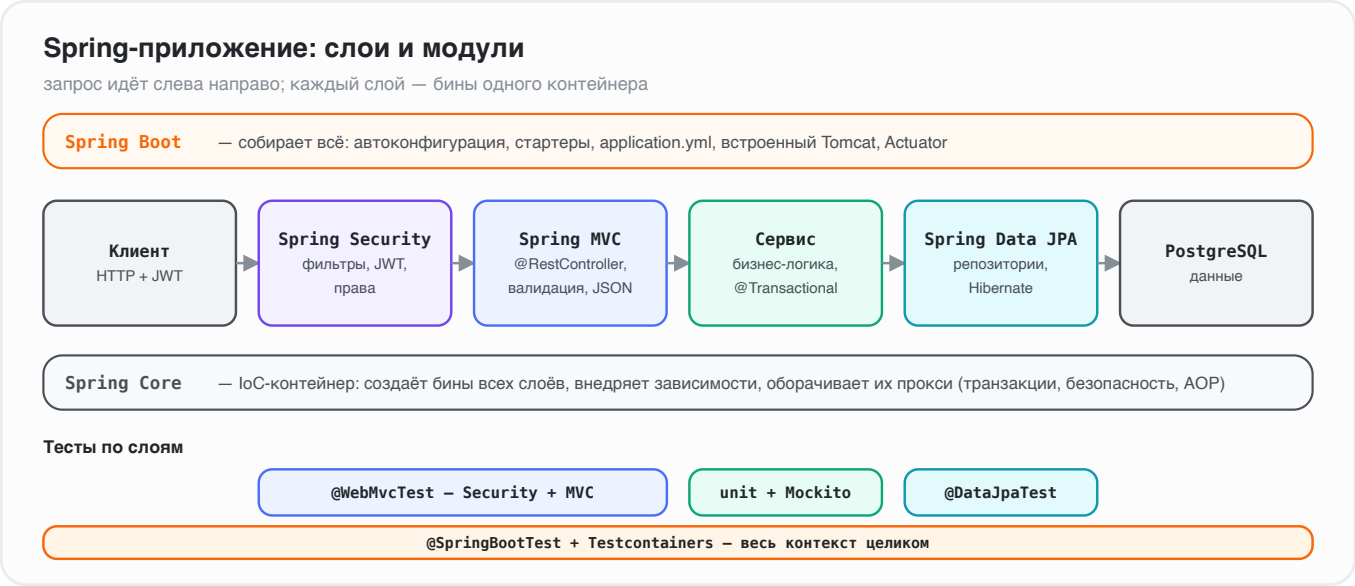


- Вступление
- Spring Core
  - IoC (Inversion of Control) Container
  - Dependency Injection и частые проблемы, @Autowired, @Lazy
  - Области видимости бинов (Bean Scope, Singleton, Prototype scope)
  - Prototype внутри singleton и scoped proxy
  - Жизненный цикл бинов (Bean Lifecycle, @PostConstruct, @PreDestroy)
  - Полный жизненный цикл и BeanPostProcessor
  - Прокси в Spring — JDK vs CGLIB
  - @Configuration vs @Component для @Bean
  - Потокобезопасность singleton-бинов
  - Циклические зависимости в Spring Boot 2.6+
  - Файлы свойств (properties), @PropertySource и @Value
  - Spring AOP (Aspect-Oriented Programming)

    - События — ApplicationEvent
- Spring Boot
    - Spring vs Spring Boot
    - Автоконфигурация
    - Стартеры и версии зависимостей
    - Конфигурация: application.yml и профили
    - @ConfigurationProperties vs @Value
    - Встроенный сервер и graceful shutdown
    - Actuator и наблюдаемость
    - Виртуальные потоки в Spring Boot
    - Что нового в Spring Boot 4 и Spring Framework 7
- Spring MVC и REST
    - DispatcherServlet и путь запроса
    - Контроллеры и маппинг
    - Валидация входных данных
    - Обработка ошибок и ProblemDetail
    - Filter vs Interceptor vs AOP
    - REST: методы, коды, идемпотентность
    - HTTP-клиенты
    - Spring MVC vs WebFlux
    - Версионирование API в Spring 7
- Spring Transactions
    - Как работает @Transactional
    - Почему @Transactional не работает
    - Откат: rollbackFor и rollback-only
    - Propagation — распространение транзакции
    - Isolation — уровни изоляции

    - readOnly и timeout
    - Программные транзакции — TransactionTemplate
    - Транзакции и внешний мир — Outbox
- Spring Data JPA
    - Репозитории Spring Data
    - Запросы: derived, @Query, проекции, пагинация
    - Проблема N+1
    - Ленивая загрузка и open-in-view
    - save(): persist или merge
    - Блокировки: optimistic и pessimistic
    - Аудит сущностей
- Spring Security
    - Архитектура: цепочка фильтров
    - Настройка SecurityFilterChain
    - Аутентификация и авторизация
    - Method security
    - JWT
    - Хранение паролей
    - CSRF и CORS
- Spring Testing
    - Виды тестов в Spring
    - Unit-тесты: JUnit 5 + Mockito
    - @WebMvcTest — тест контроллера
    - @DataJpaTest и Testcontainers
    - @SpringBootTest — интеграционный тест
    - Ловушки транзакций в тестах
- Вопросы к собеседованию

- Spring Core — вопросы
- Spring Boot — вопросы
- Spring MVC и REST — вопросы
- Spring Transactions — вопросы
- Spring Data JPA — вопросы
- Spring Security — вопросы
- Spring Testing — вопросы

Вступление

Конспект по Spring для Java-бэкенда: контейнер, Spring Boot, веб-слой, транзакции, работа с данными, безопасность и тесты. Ориентир — **Spring Boot 3.x / 4.0** и **Spring Framework 6 / 7** (Boot 4 вышел в ноябре 2025), Java 21+.

Темы идут в порядке схемы выше:

- **Spring Core** и **Spring Boot** — фундамент: как создаются бины и собирается приложение;
- **Spring MVC и REST** — как запрос попадает в контроллер и превращается в ответ;
- **Spring Transactions** и **Spring Data JPA** — сервисный слой и работа с БД;
- **Spring Security** — что происходит с запросом до контроллера;
- **Spring Testing** — как проверить каждый слой.

В конце — вопросы к собеседованию по каждой теме. Примеры кода — из платёжного домена: счета, переводы,

идемпотентность.

Spring Core

Spring Core — IoC-контейнер: создаёт объекты-бины, связывает их зависимостями и управляет жизненным циклом. На нём построены все остальные модули.

IoC (Inversion of Control) Container

IoC расшифровывается как **Inversion of Control** (на русском — **Инверсия управления**).

Это главный архитектурный принцип, на котором строится Spring и многие другие современные фреймворки.

В обычном программировании ты сам управляешь кодом: пишешь `new MyService()`, то есть сам контролируешь создание объектов. При **IoC** ты отдаешь этот контроль фреймворку. Теперь Spring решает, когда создать объект, как его настроить и куда внедрить, а твой код просто пользуется готовым результатом.

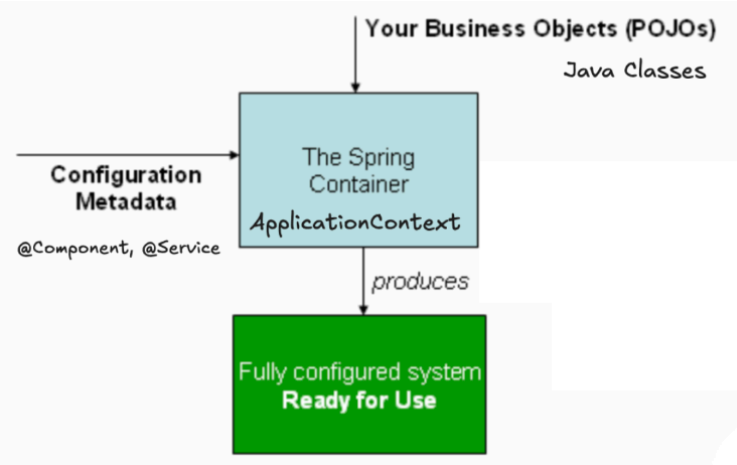
Container Overview

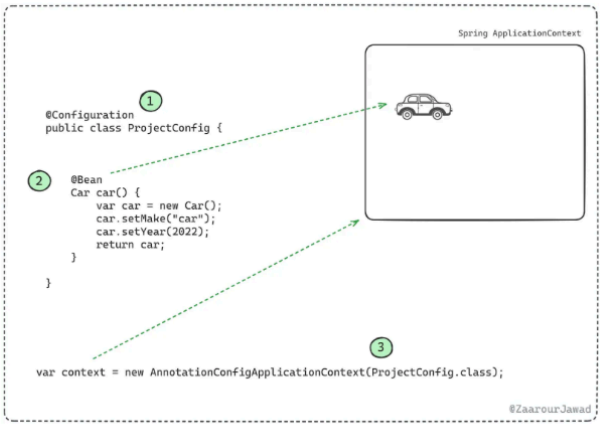
Контейнер (**ApplicationContext**) — это главный центр управления Spring. Его единственная задача — создавать, настраивать и связывать объекты вашего приложения.

Любой объект, который попал под управление этого контейнера, называется бином (**Bean**). В Spring Boot контейнер запускается автоматически при старте.

Чтобы собрать приложение, контейнеру нужны инструкции. Вы можете передать их тремя путями: через (1) аннотации прямо в коде, через (2) Java-классы конфигурации или через (3 old) XML-файлы. Для Spring все эти форматы равны.

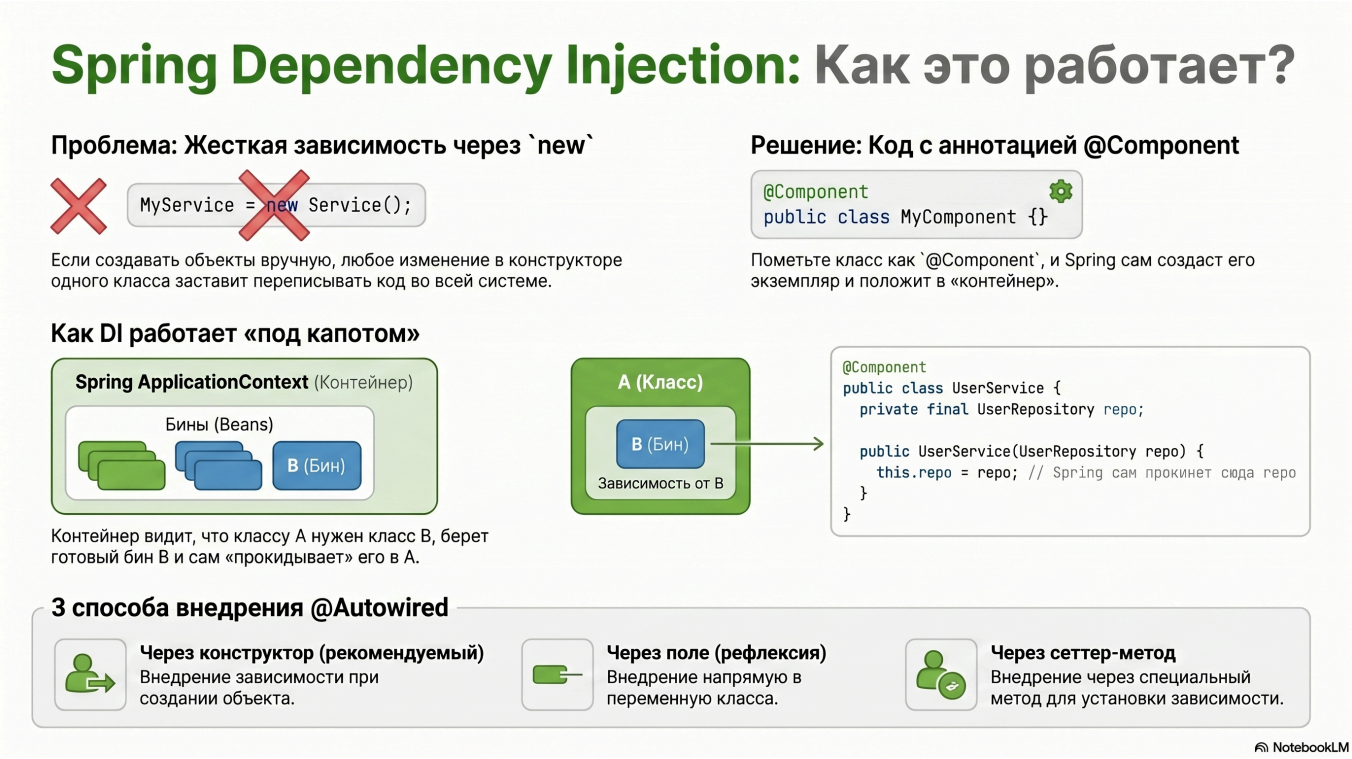
Вы можете достать любой объект вручную через метод getBean. Но документация строго запрещает делать это в рабочем коде. Приложение должно получать объекты автоматически через внедрение зависимостей, чтобы ваш код не привязывался жестко к библиотекам Spring.





Dependency Injection и частые проблемы, @Autowired, @Lazy

Способы внедрения зависимостей (Dependency Injection)



В Spring существует три основных способа внедрить один бин в другой:

- **Constructor Injection (Внедрение через конструктор):** Зависимость передается в аргументы конструктора. Это **рекомендуемый способ**, так как он позволяет делать поля `final` и облегчает тестирование. Если в классе только один конструктор, аннотацию `@Autowired` ставить необязательно — Spring поймет это автоматически.
- **Field Injection (Внедрение через поле):** Аннотация `@Autowired` ставится непосредственно над полем. Spring использует рефлексию для внедрения зависимости после создания объекта через конструктор по умолчанию. Поля в этом случае не могут быть `final`.
- **Setter Injection (Внедрение через сеттер):** Зависимость передается через метод-сеттер, помеченный `@Autowired`.

2. Частые проблемы при внедрении зависимостей и их решения

Проблема №1: Отсутствие подходящего бина Если Spring не находит бин нужного типа для внедрения, контекст не сможет запуститься и выдаст ошибку.

- **Решение:** Если зависимость необязательна, можно использовать параметр `@Autowired(required = false)`. В этом случае, если бина нет в контексте, в поле придет `null`, и приложение запустится.

Проблема №2: Неоднозначность (несколько бинов одного типа) Если в контексте есть два и более бина одного типа (например, две реализации одного интерфейса), Spring не поймет, какой именно нужно внедрить.

- **Решение 1:** `@Qualifier("beanName")`. Позволяет указать конкретное имя бина, который нужно внедрить.
- **Решение 2:** `@Primary`. Помечает один из бинов как приоритетный. При возникновении неоднозначности Spring выберет именно его.

Проблема №3: Циклическая зависимость (Circular Dependency) Ситуация, когда Бин А зависит от Бина Б, а Бин Б в свою очередь зависит от Бина А. Spring не сможет создать их через конструкторы и выдаст ошибку.

- **Решение 1: Внедрение через сеттеры**. Spring сначала создаст объекты, а затем проставит зависимости через сеттеры.

- **Решение 2: Аннотация `@Lazy`**. Spring внедрит не сам объект, а специальный **прокси-объект** (обертку). Реальный объект будет создан только при первом обращении к нему.

Области видимости бинов (Bean Scope, Singleton, Prototype scope)

Scope определяет, как и когда Spring создает экземпляры бинов.

- **Singleton (по умолчанию):** Создается **один единственный экземпляр** на весь Application Context. Все другие бины, запрашивающие эту зависимость, получат ссылку на один и тот же объект.
- **Prototype:** Каждый раз, когда мы запрашиваем бин из контекста или он внедряется в другой бин, создается **новый экземпляр**.
    - **Важная особенность:** Prototype-бины **не хранятся в Spring-контейнере** после создания. Spring только создает их и отдает разработчику, не управляя их дальнейшим жизненным циклом (например, для них не вызовутся методы уничтожения `@PreDestroy`).

Другие скопы (в основном для веб-приложений):

- **Request:** один бин на один HTTP-запрос.
- **Session:** один бин на одну пользовательскую сессию.
- **Websocket:** для работы через веб-сокеты.

Prototype внутри singleton и scoped proxy

Проблема: prototype-бин, внедрённый в singleton, создаётся **один раз** — в момент создания singleton. Дальше singleton

всегда держит тот же экземпляр.

Решения:

```
@Service
class StatementService {
    private final ObjectProvider<StatementBuilder>
builders;   // prototype

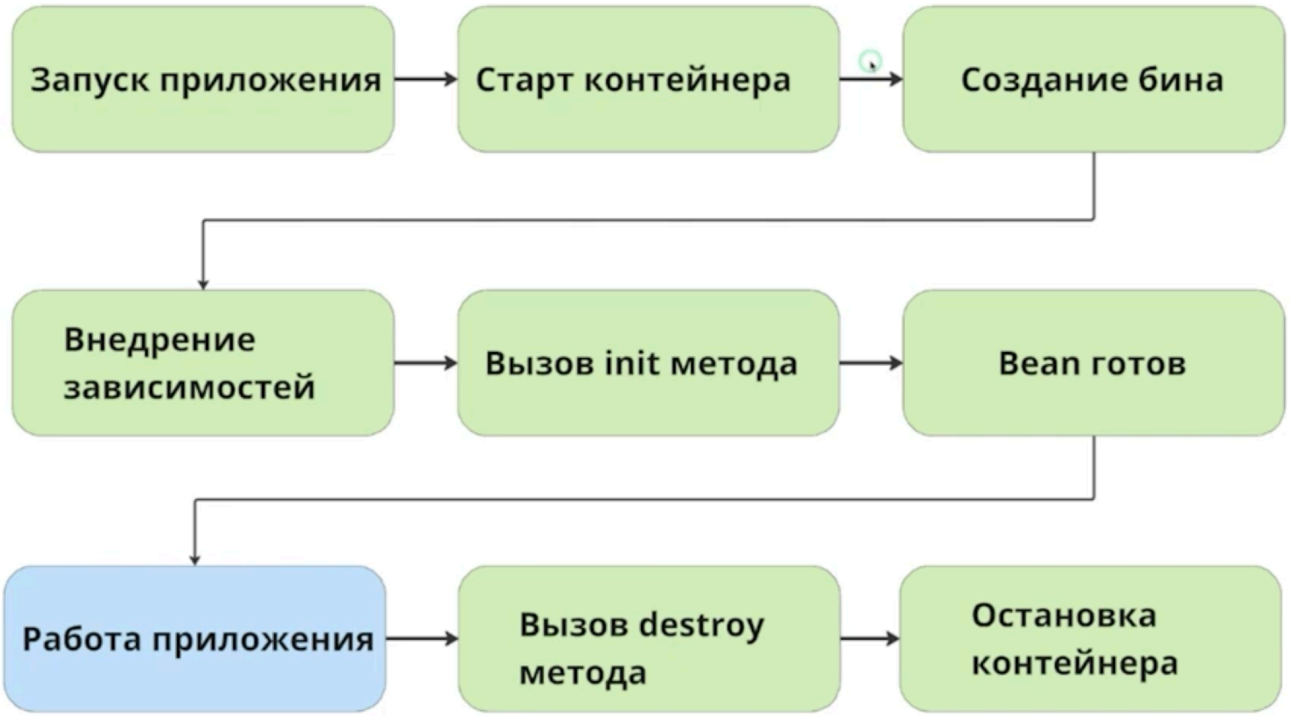
    StatementService(ObjectProvider<StatementBuilder>
builders) {
        this.builders = builders;
    }

    Statement build(UUID accountId) {
        return
builders.getObject().forAccount(accountId).build();  //
новый экземпляр на вызов
    }
}
```

- `ObjectProvider<T>` / `Provider<T>` — достаём бин из контекста при каждом вызове (рекомендуемый способ);
- `@Lookup`-метод — Spring переопределит его и будет возвращать новый бин;
- **scoped proxy** — `@Scope(value = "prototype", proxyMode = ScopedProxyMode.TARGET_CLASS)`: внедряется прокси, который на каждый вызов берёт свежий объект.

`@RequestScope` и `@SessionScope` уже включают `proxyMode = TARGET_CLASS` — поэтому request-бин можно внедрить в singleton-контроллер.

Жизненный цикл бинов (Bean Lifecycle, @PostConstruct, @PreDestroy)



Жизненный цикл бина описывает все этапы, которые проходит объект с момента запуска приложения до его уничтожения.

Основные этапы жизненного цикла:

1. **Запуск приложения и контейнера:** Spring сканирует классы и определяет, какие бины нужно создать.
2. **Создание бина:** Вызывается конструктор класса.
3. **Внедрение зависимостей:** Происходит прокидывание всех необходимых зависимостей через поля (`@Autowired` над полем) или сеттеры.
4. **Вызов Init-метода (`@PostConstruct`):** Выполняется кастомная логика настройки бина перед тем, как он будет готов к использованию.
5. **Работа бина:** Бин готов и используется в приложении.

6. **Остановка приложения:** Контейнер начинает процесс закрытия.
7. **Вызов Destroy-метода (`@PreDestroy`):** Выполняется логика перед уничтожением бина (например, закрытие ресурсов).
8. **Остановка контейнера:** Полное завершение работы после уничтожения всех бинов.

Аннотации управления циклом

Для встраивания своей логики в жизненный цикл используются специальные аннотации. Чтобы они работали в современных версиях Java, необходимо добавить зависимость `jakarta.annotation-api`.

1. @PostConstruct (Init-метод)

- **Когда вызывается:** Сразу после создания бина и внедрения в него всех зависимостей, но **до того**, как Spring отдаст бин вызывающему коду или другим сервисам.
- **Назначение:** Дополнительная настройка или проверка состояния объекта.
- **Требования:** Метод не должен принимать аргументов и должен возвращать `void`.

2. @PreDestroy (Destroy-метод)

- **Когда вызывается:** Непосредственно перед тем, как контекст закроется и бин будет уничтожен.
- **Назначение:** Освобождение ресурсов, таких как закрытие соединений с базой данных или файловых потоков.

- **Требования:** Метод также не должен иметь аргументов и должен возвращать `void`.

Различия для Singleton и Prototype

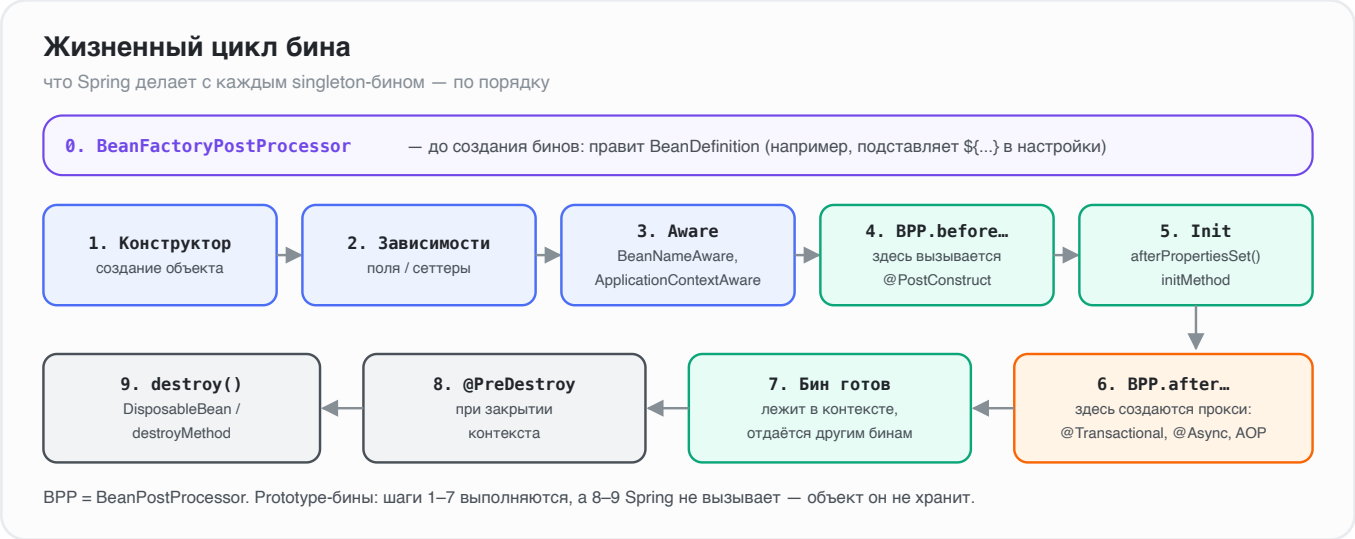
Жизненный цикл бинов существенно зависит от их **скоупа** (области видимости).

- **Singleton (по умолчанию):**
  - Spring полностью управляет жизненным циклом.
  - Создается в единственном экземпляре и **хранится в Application Context**.
  - Методы `@PostConstruct` и `@PreDestroy` **вызываются автоматически** самим Spring.
- **Prototype:**
  - Spring **не хранит ссылки** на Prototype-бины в своем контексте.
  - Каждый раз при запросе создается новый объект.
  - `@PostConstruct` **вызывается** при создании.
  - **`@PreDestroy` НЕ вызывается автоматически**, так как Spring «теряет» объект после создания. Ответственность за уничтожение и очистку ресурсов ложится на плечи **разработчика**.

Важный нюанс: Если бин имеет скоуп `Prototype`, его создание (включая `@PostConstruct`) может происходить не при старте приложения, а по требованию, в момент первого обращения к контексту.

Полный жизненный цикл и BeanPostProcessor

BeanPostProcessor (BPP) — специальный бин, который Spring вызывает для **каждого** другого бина до и после инициализации. Он может вернуть вместо бина обёртку — именно так появляются прокси для `@Transactional`, `@Async`, AOP.
BeanFactoryPostProcessor (BFPP) — работает раньше: когда ни один обычный бин ещё не создан, и меняет не объекты, а их описания (`BeanDefinition`).




```
@Component
class AuditBeanPostProcessor implements
BeanPostProcessor {

    @Override
    public Object
postProcessBeforeInitialization(Object bean, String
beanName) {
        if
(bean.getClass().isAnnotationPresent(Audited.class)) {
            System.out.println("init: " + beanName);
```

```
        }
        return bean;            // можно вернуть другой
объект — например, прокси
    }
}
```

Порядок колбэков, если заданы все сразу:

Этап	Порядок вызова
Инициализация	`@PostConstruct` → `InitializingBean.afterPropertiesSet()` → `@Bean(initMethod = …)`
Уничтожение	`@PreDestroy` → `DisposableBean.destroy()` → `@Bean(destroyMethod = …)`

> 🔥 **BFPP объявляют через `static @Bean`**
>
> Иначе Spring создаст конфигурационный класс слишком рано, и часть BPP к нему не применится.

Прокси в Spring — JDK vs CGLIB

Прокси — объект-обёртка того же типа, что и бин: перехватывает вызов, делает свою работу (открыть транзакцию, проверить права, засечь время) и передаёт вызов настоящему объекту.

	JDK Dynamic Proxy	CGLIB
Как устроен	реализует интерфейсы	наследует класс бина

	JDK Dynamic Proxy	CGLIB
	бина	
Нужен интерфейс	да	нет
Что не перехватит	методы вне интерфейса	`final`-методы, `private`-методы; `final`-класс не проксируется вообще
В Spring Boot	—	по умолчанию (`spring.aop.proxy-target-class=true`)

Отсюда три следствия, о которых спрашивают на собеседованиях:

- **self-invocation** — вызов `this.method()` внутри бина идёт мимо прокси, аннотация не сработает;
- аннотации на `private`-методах игнорируются;
- в Spring 6+ `@Transactional` с CGLIB-прокси работает и на `protected`/package-private методах, но не на `private`.

```
@Service
class ReportService {
    private final ReportService self;           // ссылка на прокси самого себя

    ReportService(@Lazy ReportService self) { this.self = self; }

    public void rebuildAll() {
        self.rebuild();                          // ✅
```

```
через прокси — транзакция откроется
        // this.rebuild();                    // ❌
мимо прокси
    }

    @Transactional
    public void rebuild() { /* ... */ }
}
```

Лучшее решение — вынести метод в отдельный бин; самоинъекция и `TransactionTemplate` — запасные варианты.

@Configuration vs @Component для @Bean

`@Configuration` — класс-источник бинов, который Spring оборачивает CGLIB-прокси: вызов `@Bean`-метода из другого `@Bean`-метода возвращает **тот же singleton из контекста**. **`@Component` с `@Bean`** («lite mode») — без прокси: такой вызов — обычный вызов Java-метода, он создаст **новый объект** в обход контейнера.

```
@Configuration                                  //
proxyBeanMethods = true по умолчанию
class ClientsConfig {

    @Bean
    FxClient fxClient() { return new
FxClient("https://fx.example.com"); }

    @Bean
    PaymentService paymentService() {
        return new PaymentService(fxClient());   // тот
```

```
же бин fxClient из контекста
    }
}
```

`@Configuration(proxyBeanMethods = false)` — без прокси, стартует быстрее; зависимости тогда передают параметрами `@Bean`-метода. Так написаны автоконфигурации Spring Boot.

```
@Configuration(proxyBeanMethods = false)
class ClientsConfig {
    @Bean FxClient fxClient() { return new
FxClient("https://fx.example.com"); }
    @Bean PaymentService paymentService(FxClient
fxClient) { return new PaymentService(fxClient); }
}
```

Потокобезопасность singleton-бинов

Singleton — один объект на весь контекст, его одновременно используют все потоки (все HTTP-запросы). Spring **не делает** его потокобезопасным — это задача кода.

```
@Service
class RateService {
    private BigDecimal lastRate;                    // ❌
общее изменяемое поле — гонка между запросами

    BigDecimal convert(BigDecimal amount, BigDecimal
rate) {
        lastRate = rate;
        return amount.multiply(lastRate);
```

```
    }
}
```

Правило: бин без изменяемого состояния (stateless). Если состояние нужно:

- данные запроса — в локальных переменных и параметрах;
- общий кэш/счётчик — `ConcurrentHashMap`, `AtomicLong`, `LongAdder`;
- состояние на запрос — `@RequestScope`-бин;
- `ThreadLocal` — осторожно: с пулами и виртуальными потоками его нужно очищать.

Циклические зависимости в Spring Boot 2.6+

Циклическая зависимость — бин A зависит от B, а B от A (напрямую или через цепочку).

С **Spring Boot 2.6** циклы **запрещены по умолчанию** — приложение не стартует, даже при внедрении через поля или сеттеры:

```
The dependencies of some of the beans in the
application context form a cycle:
┌──────┐
|  paymentService
↑      ↓
|  notificationService
└──────┘
```

Решение	Комментарий
Перепроектировать	вынести общую логику в третий бин или заменить вызов на событие — правильный путь
`@Lazy` на одной зависимости	внедряется прокси, настоящий бин создаётся при первом вызове
`spring.main.allow-circular-references=true`	возвращает старое поведение — костыль

> ⚠ **Уточнение к разделу про DI**
>
> Совет «разорвать цикл сеттерами» работает только при `allow-circular-references=true`. В современном Spring Boot цикл — сигнал, что пора менять дизайн.

Файлы свойств (properties), @PropertySource и @Value

Использование конфигурационных файлов позволяет отделить программный код от настроек приложения. Записывать настройки напрямую в коде (хардкодить) — плохая практика, так как это связывает код и конфигурацию, затрудняя запуск приложения в разных окружениях (локально, тест, прод).

Аннотация @Value

Используется для внедрения значений в поля бина.

- **Прямое внедрение:** Можно указать конкретное значение прямо в аннотации, например `@Value("MainTask")`.

- **Внедрение из файлов свойств:** Используется синтаксис `${ключ}`, чтобы Spring подставил значение из подключенного файла.
    - *Пример:* `@Value("${task.name}") private String name;`.

Внедрение сложных структур данных

С помощью Spring Expression Language (SpEL) и аннотации `@Value` можно внедрять не только строки и числа, но и коллекции:

- **List и Set:** Для внедрения списка значений из строки (например, через запятую) используется синтаксис с решеткой: `#{'${list.values}'.split(',')}`.
- **Map:** Можно внедрять мапы, где ключи и значения разделены специальными символами в файле свойств.

Подключение файлов через @PropertySource

Чтобы Spring знал, из какого файла брать значения для `${...}`, необходимо указать путь к этому файлу.

- **Аннотация @PropertySource:** Вешается над конфигурационным классом (помеченным `@Configuration`).
- **Синтаксис:** `@PropertySource("classpath:application.properties")` указывает Spring искать файл `application.properties` в папке ресурсов (resources).

Работа с разными окружениями (Профили)

Для решения проблемы разных настроек для разных сред (например, база данных для разработчика и база данных для продакшена) используются **профили**.

1. Создаются разные файлы свойств для каждого окружения (например, `application-dev.properties`, `application-test.properties`, `application-prod.properties`).
2. При запуске приложения указывается активный профиль (например, через параметр запуска `java -Dspring.profiles.active=prod`).
3. Spring автоматически подтягивает нужный файл конфигурации, соответствующий выбранному профилю.

Task Properties как отдельный компонент

Хорошим подходом является создание отдельного бина (например, с аннотацией `@Component`), который будет содержать все необходимые настройки. Это позволяет инжектить объект с настройками в другие сервисы вместо того, чтобы расставлять `@Value` над полями в каждом классе.

Spring AOP (Aspect-Oriented Programming)

Spring AOP — это парадигма программирования, позволяющая разделять основную бизнес-логику и **сквозную функциональность** (logging, транзакции, проверка прав доступа, сбор метрик).

Проблема, которую решает AOP: без этого механизма технический код (например, открытие транзакции или

логирование) пришлось бы дублировать в каждом методе каждого сервиса, что ведет к сильной связанности и запутанности кода.

> **Info**
>
> В реальной практике, вероятнее всего не придется самому часто писать aop, тк в спринге есть готовые решения, но полезно знать как работает

Механизм работы: Паттерн «Прокси»

Реализация AOP в Spring строится на использовании **динамических прокси**.

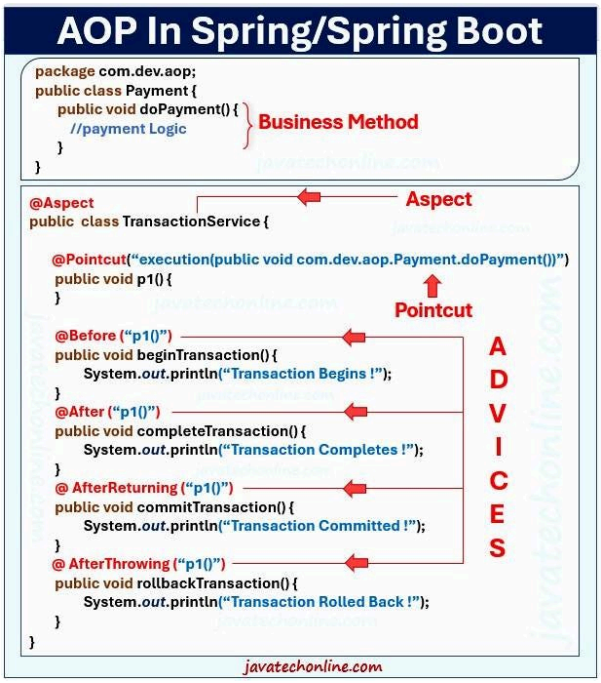
- Вызывающий код взаимодействует не с самим целевым объектом (бином), а с его **оберткой (прокси)**.
- Прокси-объект сначала выполняет вспомогательную логику, а затем делегирует выполнение самому сервису.
- При включении AOP в контекст Spring вместо обычного бина кладется его прокси-версия (например, созданная с помощью библиотеки `CGLIB`).

Основные понятия AOP

- **Аспект (Aspect):** Модульный класс, содержащий сквозную функциональность. Помечается аннотацией `@Aspect`.
- **Эдвайс (Advice):** Конкретное действие, которое выполняется

в определенной точке (до, после или вместо метода).

- **Joint Point (Точка соединения):** Точка в программе (например, вызов метода), где может быть применен аспект.
- **Pointcut (Срез):** Выражение, определяющее, к каким именно методам или классам нужно применить конкретный эдвайс.



Типы эдвайсов (Advice Types)

Spring AOP предлагает несколько типов аннотаций для управления моментом выполнения логики:

1. `@Before`: Выполняется **до** вызова целевого метода.
2. `@AfterReturning`: Отрабатывает **после успешного** завершения метода. Позволяет получить доступ к возвращаемому результату.
3. `@AfterThrowing`: Выполняется, если метод **выбросил исключение**. Позволяет перехватить и обработать ошибку.

4. **`@After`:** Отрабатывает **всегда** после завершения метода (аналог блока `finally`), независимо от того, возникла ошибка или нет.
5. **`@Around`:** Самый мощный эдвайс. Полностью **оборачивает** вызов метода. Позволяет выполнять код до и после вызова, замерять время выполнения, обрабатывать исключения и даже подменять возвращаемое значение.

Практическая реализация

Для работы AOP в проекте необходимо:

- Подключить зависимости `spring-aop` и `aspectjweaver`.
- Активировать механизм с помощью аннотации **`@EnableAspectJAutoProxy`** над конфигурационным классом.
- Создать класс-аспект, помеченный `@Aspect` и `@Component`.
- В методах эдвайсов можно использовать объект **`JoinPoint`** (или `ProceedingJoinPoint` для `@Around`) для получения метаданных о методе (имя, аргументы).

Использование кастомных аннотаций

Часто аспекты применяют не ко всем методам пакета, а только к помеченным специальной аннотацией (например, `@Loggable`).

- В Pointcut указывается путь к аннотации через **`@annotation(название_параметра)`**.
- Это позволяет гибко настраивать поведение аспектов, передавая параметры через поля аннотации (например,

уровень логирования или количество повторений).

Многие стандартные возможности Spring (например, аннотация `@Transactional`) работают именно через механизмы AOP под капотом.

События — ApplicationEvent

Событие — объект, который один бин публикует, а другие получают через `@EventListener`, не зная друг о друге. Это ослабляет связность: платёжный сервис не зависит от сервиса уведомлений.

```
public record PaymentCompleted(UUID paymentId,
BigDecimal amount) {}

@Service
class PaymentService {
    private final ApplicationEventPublisher events;
    PaymentService(ApplicationEventPublisher events) {
this.events = events; }

    @Transactional
    public void complete(Payment p) {
        p.markCompleted();
        events.publishEvent(new
PaymentCompleted(p.getId(), p.getAmount()));
    }
}

@Component
class NotificationListener {
    @TransactionalEventListener(phase =
```

```
TransactionPhase.AFTER_COMMIT)
    void on(PaymentCompleted e) { /* отправить push — только если транзакция закоммитилась */ }
}
```

- `@EventListener` — синхронно, в том же потоке и в той же транзакции;
- `@TransactionalEventListener(AFTER_COMMIT)` — только после успешного коммита;
- `@Async` на слушателе — в другом потоке.

> ⚠ **Событие после коммита — не гарантия доставки**
>
> Если приложение упадёт между коммитом и обработкой, событие потеряется. Для гарантии — паттерн Outbox (см. «Spring Transactions»).

Spring Boot

Spring Boot — способ быстро собрать приложение на Spring: автоконфигурация, стартеры, внешняя конфигурация, встроенный сервер.

Spring vs Spring Boot

Spring Framework — набор модулей: IoC-контейнер, MVC, транзакции, доступ к данным. Всё нужно подключать и настраивать вручную.
Spring Boot — надстройка над Spring: собирает приложение с разумными настройками по умолчанию. Даёт автоконфигурацию,

стартеры, встроенный веб-сервер, внешнюю конфигурацию и Actuator.

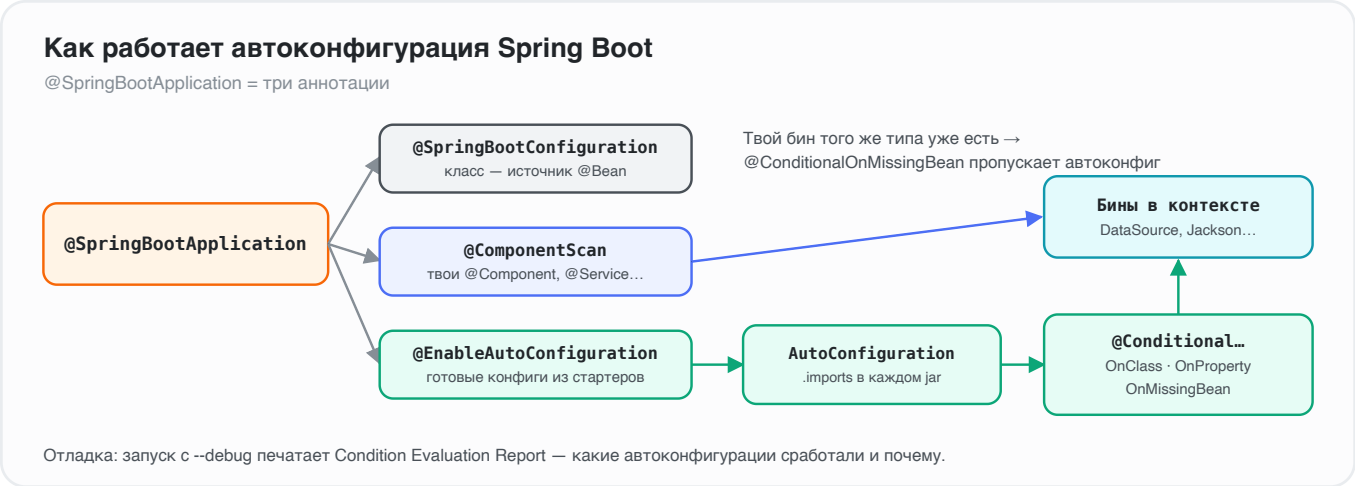
```
@SpringBootApplication
public class PaymentsApplication {
    public static void main(String[] args) {

SpringApplication.run(PaymentsApplication.class, args);
    }
}
```

Что делает `SpringApplication.run`: создаёт `ApplicationContext`, загружает настройки, применяет автоконфигурацию, поднимает встроенный Tomcat и запускает `CommandLineRunner` / `ApplicationRunner`.

Автоконфигурация

Автоконфигурация — готовые `@Configuration`-классы из jar-ов Spring Boot. Они создают бины, только если выполнены условия: нужный класс есть в classpath, задано свойство, такого бина ещё нет.



Как написать свою (например, общий клиент антифрода для нескольких сервисов):

```
@AutoConfiguration
@ConditionalOnClass(FraudClient.class)
@EnableConfigurationProperties(FraudProperties.class)
public class FraudAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean                    //
если в приложении свой FraudClient — этот не создаётся
    FraudClient fraudClient(FraudProperties props) {
        return new FraudClient(props.url());
    }
}
```

Класс регистрируется строкой в файле `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`.
Старый способ через `spring.factories` для автоконфигураций убран в Spring Boot 3.

Аннотация	**Когда бин создаётся**
`@ConditionalOnClass`	класс есть в classpath (подключена библиотека)
`@ConditionalOnMissingBean`	такого бина ещё нет — так твой бин «побеждает» автоконфиг
`@ConditionalOnProperty`	задано свойство, например `feature.fraud.enabled=true`

Аннотация	Когда бин создаётся
`@ConditionalOnWebApplication`	приложение веб

Отключить автоконфигурацию: `@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)` или `spring.autoconfigure.exclude=...`.
Понять, почему бин создался или нет: запуск с `--debug` (Condition Evaluation Report) или `/actuator/conditions`.

Стартеры и версии зависимостей

Стартер — зависимость-«набор»: подтягивает нужные библиотеки и их автоконфигурацию. Версии всех библиотек согласованы в BOM Spring Boot, поэтому в `pom.xml` их не пишут.

```
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.0.0</version>
</parent>

<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-
webmvc</artifactId>    <!-- без <version> -->
    </dependency>
</dependencies>
```

Частые стартеры бэкенда: `webmvc`, `data-jpa`, `validation`, `security`, `security-oauth2-resource-server`, `actuator`, `test`; для Kafka — зависимость `spring-kafka`.

ⓘ Переименования в Spring Boot 4

`spring-boot-starter-web` → `spring-boot-starter-webmvc`, `…-oauth2-resource-server` → `…-security-oauth2-resource-server`, `…-aop` → `…-aspectj`. Старые имена пока работают, но помечены устаревшими — в проектах на Boot 3 встретишь именно их.

Конфигурация: application.yml и профили

Внешняя конфигурация — настройки вне кода: один jar работает в dev, test и prod с разными значениями.

```yaml
server:
  port: 8080
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/payments
    username: app
    password: ${DB_PASSWORD}          # из переменной окружения
  jpa:
    open-in-view: false
payments:
  fraud-url: https://fraud.internal
  timeout: 2s
  max-retries: 3
```

Профиль — именованный набор настроек и бинов: `application-dev.yml`, `application-prod.yml`; включается

через `spring.profiles.active=prod`. Бин только для профиля — `@Profile("dev")`.

Приоритет источников (упрощённо, сверху — сильнее):

1. аргументы командной строки `--server.port=9090`;
2. системные свойства Java `-Dserver.port=9090`;
3. переменные окружения `SERVER_PORT=9090`;
4. `application-{profile}.yml`;
5. `application.yml`.

> ⚠ **Секреты — не в репозиторий**
>
> Пароли и ключи передают через переменные окружения или хранилище секретов (Vault, Kubernetes Secrets), а не коммитят в `application.yml`.

@ConfigurationProperties vs @Value

`@ConfigurationProperties` — связывает группу настроек с типизированным объектом. **`@Value`** — внедряет одно значение по ключу.

```
@ConfigurationProperties(prefix = "payments")
@Validated
public record PaymentsProperties(
        @NotNull URI fraudUrl,
        @DefaultValue("30s") Duration timeout,
        @Min(1) int maxRetries) {}

@SpringBootApplication
```

```
@ConfigurationPropertiesScan                          //
найдёт все @ConfigurationProperties
public class PaymentsApplication { /* ... */ }
```

```
@Value("${payments.max-retries:3}")                    //
одно значение, 3 — по умолчанию
private int maxRetries;
```

	`@ConfigurationProperties`	`@Value`
Тип	объект/record, вложенные структуры	одно значение
Валидация	`@Validated` + Bean Validation — ошибка при старте	нет
Гибкое связывание	`max-retries`, `maxRetries`, `MAX_RETRIES` — одно и то же	ключ пишется точно
SpEL	нет	да: `#{...}`
Когда брать	настройки модуля	одиночное значение

Встроенный сервер и graceful shutdown

Встроенный сервер — Tomcat (или Jetty) внутри jar: приложение запускается как `java -jar app.jar`, отдельный сервер приложений не нужен.

Graceful shutdown — при остановке сервер перестаёт принимать новые запросы и дожидается текущих. С **Spring Boot 3.4 включён по умолчанию**.

```yaml
spring:
  lifecycle:
    timeout-per-shutdown-phase: 30s     # сколько ждать
незавершённые запросы
```

Это важно при деплое в Kubernetes: старый под не обрывает платежи на полпути.

Actuator и наблюдаемость

Actuator — готовые служебные эндпоинты: состояние приложения, метрики, настройки логов.

Эндпоинт	Зачем
`/actuator/health`	жив ли сервис и его зависимости (БД, Kafka)
`/actuator/health/liveness`	живо ли приложение — иначе Kubernetes перезапустит под
`/actuator/health/readiness`	готово ли принимать трафик — иначе под убирают из балансировки
`/actuator/prometheus`	метрики для Prometheus (Micrometer)
`/actuator/loggers`	поменять уровень логов на лету

```yaml
management:
  endpoints:
    web:
      exposure:
```

```
        include: health, info, prometheus   # наружу —
только нужное
```

Своя метрика через Micrometer:

```
@Service
class PaymentService {
    private final Counter failed;

    PaymentService(MeterRegistry registry) {
        this.failed =
registry.counter("payments.failed");
    }

    void onFailure() { failed.increment(); }
}
```

> ⚠ **Не открывай всё**
>
> `env`, `heapdump`, `configprops` раскрывают секреты и память процесса. Наружу — только `health` и метрики, остальное — за авторизацией или на отдельном порту `management.server.port`.

Виртуальные потоки в Spring Boot

Виртуальные потоки (Java 21+) — лёгкие потоки JVM: блокирующий код на них масштабируется почти как реактивный.

```
spring:
  threads:
    virtual:
```

```
      enabled: true      # Spring Boot 3.2+: Tomcat,
@Async и планировщик — на виртуальных потоках
```

Что учитывать:

- пул соединений к БД (HikariCP) остаётся узким местом — тысячи виртуальных потоков ждут 10–20 соединений;
- до Java 24 `synchronized` «прикалывал» виртуальный поток к платформенному (pinning); в Java 24 это исправлено (JEP 491);
- `ThreadLocal`-кэши на миллионах потоков расходуют память.

Что нового в Spring Boot 4 и Spring Framework 7

Вышли в **ноябре 2025**:

- Java 17 — минимум, первоклассная поддержка Java 25;
- спецификации Jakarta EE 11: Servlet 6.1, Persistence 3.2, Validation 3.1;
- код Spring Boot разбит на более мелкие модули;
- null-safety через аннотации JSpecify;
- **версионирование API** из коробки (`@GetMapping(path = "...", version = "1.1")`);
- **HTTP service clients** — интерфейсы с `@HttpExchange`, реализацию генерирует Spring, Boot настраивает их через свойства;
- Jackson 3 по умолчанию, поддержка Jackson 2 оставлена как устаревшая;

- Spring Security 7, Tomcat 11, Jetty 12.1, стартер для OpenTelemetry.

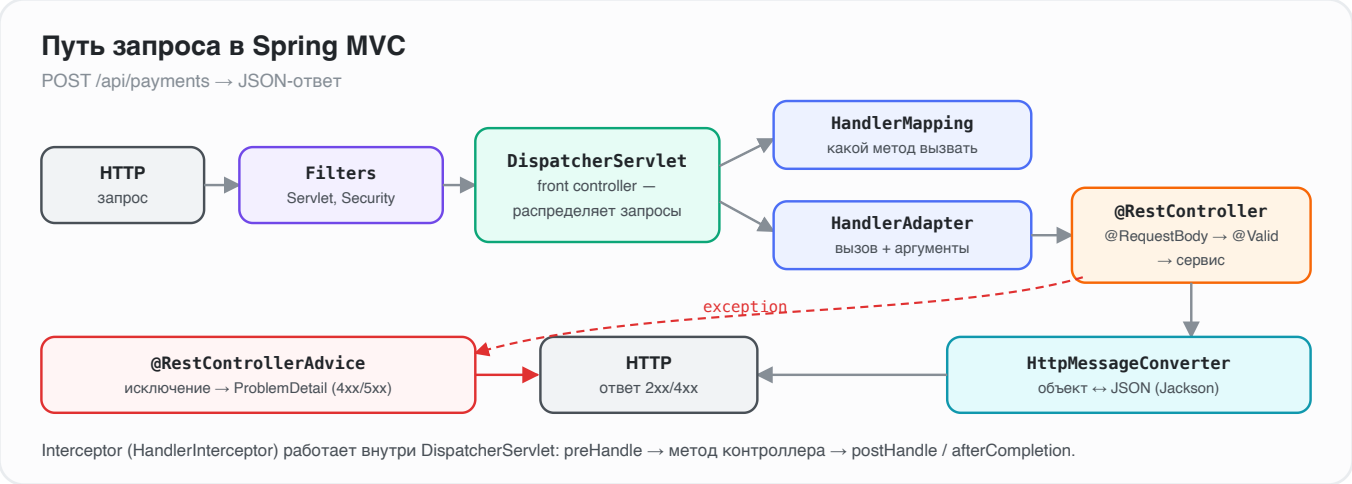
Из Spring Boot 3.4: `@MockBean` и `@SpyBean` устарели — вместо них `@MockitoBean` и `@MockitoSpyBean`.

Spring MVC и REST

Spring MVC — веб-слой приложения: принимает HTTP-запросы, вызывает сервисы и отдаёт JSON.

DispatcherServlet и путь запроса

Spring MVC — веб-фреймворк на сервлетах, модель «поток на запрос».
DispatcherServlet — front controller: единая точка входа, которая находит нужный метод контроллера, вызывает его и превращает результат в HTTP-ответ.



1. Запрос проходит **Servlet-фильтры**, в том числе Spring Security.
2. `DispatcherServlet` спрашивает `HandlerMapping`, какой

метод обрабатывает `POST /api/payments`.

3. `HandlerAdapter` готовит аргументы: JSON → объект (`HttpMessageConverter`, Jackson), валидация, `@PathVariable`.
4. Вызываются `HandlerInterceptor.preHandle` → метод контроллера → `postHandle`.
5. Возвращённый объект сериализуется в JSON. Если вылетело исключение — его обрабатывает `@RestControllerAdvice`.

Контроллеры и маппинг

`@RestController` = `@Controller` + `@ResponseBody`: результат метода пишется прямо в тело ответа (JSON), без шаблонов.

```
public record CreatePaymentRequest(UUID fromAccount,
UUID toAccount, BigDecimal amount, String currency) {}
public record PaymentResponse(UUID id, String status,
BigDecimal amount) {}

@RestController
@RequestMapping("/api/payments")
class PaymentController {
    private final PaymentService service;

    PaymentController(PaymentService service) {
this.service = service; }

    @PostMapping
    ResponseEntity<PaymentResponse>
create(@RequestHeader("Idempotency-Key") String key,
                                        @Valid
@RequestBody CreatePaymentRequest req) {
```

```
        PaymentResponse p = service.create(key, req);
        return
ResponseEntity.created(URI.create("/api/payments/" +
p.id())).body(p);   // 201 + Location
    }

    @GetMapping("/{id}")
    PaymentResponse get(@PathVariable UUID id) {
        return service.get(id);
// 200
    }

    @GetMapping
    PagedModel<PaymentResponse> list(@RequestParam UUID
accountId, Pageable pageable) {  // ?accountId=…
&page=0&size=20
        return new PagedModel<>(service.list(accountId,
pageable));   // стабильный JSON вместо сырого Page
    }
}
```

Аннотация	Откуда берёт данные
`@PathVariable`	часть пути: `/payments/{id}`
`@RequestParam`	query-параметр: `?accountId=…`
`@RequestBody`	тело запроса (JSON → объект)
`@RequestHeader`	заголовок
`ResponseEntity<T>`	полный контроль над статусом, заголовками и телом

Валидация входных данных

Bean Validation (Jakarta Validation, реализация — Hibernate Validator) — проверка объектов по аннотациям-ограничениям. Нужен стартер `spring-boot-starter-validation`.

```
public record CreatePaymentRequest(
        @NotNull UUID fromAccount,
        @NotNull UUID toAccount,
        @NotNull @Positive @Digits(integer = 12,
fraction = 2) BigDecimal amount,
        @NotBlank @Size(min = 3, max = 3) String
currency) {}
```

- `@Valid` на `@RequestBody` → при ошибке `MethodArgumentNotValidException` → **400**;
- ограничения прямо на параметрах (`@PathVariable @Positive long id`) Spring 6.1+ проверяет встроенной валидацией методов → `HandlerMethodValidationException`;
- для вложенных объектов — `@Valid` на поле.

> **Деньги — только BigDecimal**
>
> `double` теряет точность (`0.1 + 0.2 != 0.3`). Для сумм — `BigDecimal` и `@Digits`, в БД — `numeric(14,2)`.

Обработка ошибок и ProblemDetail

`@RestControllerAdvice` — глобальный обработчик исключений для всех контроллеров.
`ProblemDetail` — стандартный формат ошибки по RFC 9457 (`application/problem+json`), есть в Spring 6+.

```
@RestControllerAdvice
class ApiExceptionHandler {

    @ExceptionHandler(PaymentNotFoundException.class)
    ProblemDetail notFound(PaymentNotFoundException e)
{
        ProblemDetail pd =
ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND,
e.getMessage());
        pd.setTitle("Payment not found");
        return pd;
    }

    @ExceptionHandler(InsufficientFundsException.class)
    ProblemDetail
insufficientFunds(InsufficientFundsException e) {
        return
ProblemDetail.forStatusAndDetail(HttpStatus.UNPROCESSAB
LE_ENTITY, e.getMessage());
    }
}
```

```
{ "type": "about:blank", "title": "Payment not found",
"status": 404,
  "detail": "Payment 7f3c… not found", "instance":
"/api/payments/7f3c…" }
```

`spring.mvc.problemdetails.enabled=true` — встроенные ошибки Spring (400 валидации, 405 и др.) тоже будут в этом формате.

Filter vs Interceptor vs AOP

	Servlet Filter	HandlerInterceptor	AOP (`@Aspect`)
Уровень	Servlet-контейнер, до Spring MVC	внутри `DispatcherServlet`	любой бин Spring
Видит	сырой запрос/ответ	запрос + какой метод контроллера вызовется	аргументы и результат метода
Типичное применение	безопасность, CORS, логирование, request-id	проверка заголовков, метрики по эндпоинтам	транзакции, кэш, аудит сервисов

REST: методы, коды, идемпотентность

REST — стиль API: ресурсы (`/payments/{id}`), HTTP-методы как действия, состояние не хранится между запросами.

Метод	Назначение	Безопасный	Идемпотентный
`GET`	прочитать	да	да
`POST`	создать / действие	нет	**нет**
`PUT`	заменить целиком	нет	да
`PATCH`	изменить часть	нет	не обязательно
`DELETE`	удалить	нет	да

Коды ответа: `200` OK · `201` Created · `204` No Content · `400` неверный запрос · `401` не аутентифицирован · `403` нет прав · `404` не найдено · `409` конфликт · `422` бизнес-ошибка · `429`

слишком много запросов · `500` ошибка сервера · `503` сервис недоступен.

> ⚠ **POST платежа нужно сделать идемпотентным**
>
> Клиент повторит запрос при таймауте — без защиты спишутся деньги дважды. Клиент шлёт `Idempotency-Key`, сервер сохраняет ключ с уникальным индексом и на повтор отдаёт прежний ответ.

HTTP-клиенты

Клиент	Статус	Когда
`RestTemplate`	поддерживается, новых функций нет	старый код
`RestClient`	Spring 6.1+, синхронный, fluent API	обычный синхронный вызов
`WebClient`	реактивный	WebFlux, неблокирующий код
HTTP interface (`@HttpExchange`)	Spring 6+, в Boot 4 — автоконфигурация	декларативный клиент: только интерфейс

```
RestClient fx = RestClient.builder()
        .baseUrl("https://fx.example.com")
        .build();

Rate rate = fx.get()
        .uri("/rates/{pair}", "EURUSD")
```

```
            .retrieve()
            .body(Rate.class);
```

```
@HttpExchange("/rates")
interface FxClient {
    @GetExchange("/{pair}")
    Rate rate(@PathVariable String pair);     //
реализацию сгенерирует Spring
}
```

> ⚠ **Всегда задавай таймауты**
>
> Без таймаута зависший внешний сервис занимает потоки, пока не встанет всё приложение. Плюс ретраи с backoff и circuit breaker (Resilience4j).

Spring MVC vs WebFlux

	Spring MVC	**Spring WebFlux**
Модель	поток на запрос (Tomcat)	event loop, мало потоков (Netty)
Код	обычный, блокирующий	реактивный: `Mono`, `Flux`
Драйвер БД	JDBC / JPA	R2DBC
Сложность	низкая	выше: отладка, стектрейсы
Когда	большинство бэкендов	очень много одновременных соединений, стриминг

С виртуальными потоками (`spring.threads.virtual.enabled=true`) MVC держит большую конкурентную нагрузку, оставаясь блокирующим, — WebFlux нужен реже.

Версионирование API в Spring 7

Версионирование API — несколько версий одного эндпоинта одновременно, чтобы не ломать старых клиентов. В Spring Framework 7 / Boot 4 есть из коробки.

```
@RestController
class AccountController {

    @GetMapping(path = "/accounts/{id}", version =
"1.0")
    AccountV1 getV1(@PathVariable UUID id) { /* ... */
}

    @GetMapping(path = "/accounts/{id}", version =
"1.1")
    AccountV2 getV2(@PathVariable UUID id) { /* ... */
}
}
```

```
spring.mvc.apiversion.use.header=API-Version     #
версия из заголовка API-Version: 1.1
```

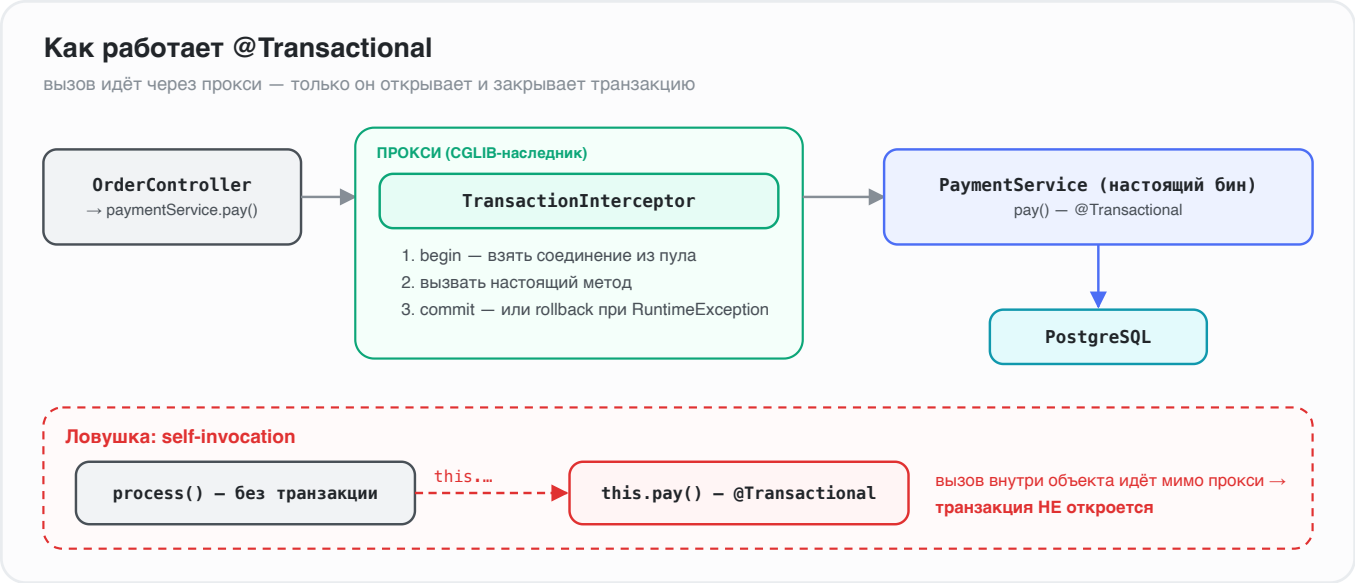
До Spring 7 версию обычно клали в путь вручную: `/api/v1/...`, `/api/v2/...`.

Spring Transactions

Транзакции в Spring — декларативное управление транзакциями БД через прокси и `@Transactional`.

Как работает @Transactional

Транзакция — группа операций с БД, которая выполняется целиком или не выполняется вовсе (ACID).
`@Transactional` — декларативное управление транзакцией: Spring оборачивает бин прокси, а прокси через `TransactionInterceptor` и `PlatformTransactionManager` (с JPA — `JpaTransactionManager`) открывает транзакцию до метода и фиксирует или откатывает её после.




```
@Service
public class TransferService {
    private final AccountRepository accounts;

    public TransferService(AccountRepository accounts)
{ this.accounts = accounts; }
```

```
    @Transactional
    public void transfer(UUID from, UUID to, BigDecimal
amount) {
        Account a =
accounts.findByIdForUpdate(from).orElseThrow();
        Account b =
accounts.findByIdForUpdate(to).orElseThrow();
        a.withdraw(amount);        // нет денег →
InsufficientFundsException (RuntimeException) →
rollback
        b.deposit(amount);
    }                              // commit: dirty
checking сам сохранит оба счёта, save() не нужен
}
```

- Соединение с БД привязывается к **текущему потоку** (`TransactionSynchronizationManager`, внутри `ThreadLocal`) — все репозитории в этом потоке работают в одной транзакции.
- Аннотацию ставят на **сервисный слой**; на классе — действует на все его public-методы.
- Есть два варианта: `org.springframework…Transactional` и `jakarta.transaction.Transactional`. Spring понимает оба, но у своей больше настроек: propagation, isolation, readOnly.

Почему @Transactional не работает

Причина	Что происходит
Self-invocation — `this.pay()` из того же класса	вызов мимо прокси, транзакции нет

Причина	Что происходит
`private`-метод	прокси его не перехватывает (с CGLIB в Spring 6+ работают `protected` и package-private)
`final`-метод или класс	CGLIB не может переопределить
Объект создан через `new`	это не бин, прокси нет
Исключение поймано внутри метода	прокси его не видит → **commit**
Checked-исключение	по умолчанию **commit** (см. «Откат»)
Работа в другом потоке: `@Async`, `CompletableFuture`, parallel stream	транзакция привязана к потоку, в новый поток не переходит
`@Transactional` на `@PostConstruct`-методе	прокси ещё не создан — метод вызывается на самом объекте

```
@Service
class PaymentService {
    public void process(Payment p) {
        validate(p);
        this.pay(p);                // ❌ мимо прокси — транзакции нет
    }

    @Transactional
    public void pay(Payment p) { /* ... */ }
}
```

Как исправить self-invocation, от лучшего к худшему: вынести `pay()` в отдельный бин → `TransactionTemplate` (см. «Программные транзакции») → внедрить прокси самого себя через `@Lazy`.

Откат: rollbackFor и rollback-only

Правило по умолчанию: откат при `RuntimeException` и `Error`; при **checked**-исключении — **commit**.

```
@Transactional(rollbackFor = Exception.class)
// откат и на checked
public void importStatement(Path file) throws
IOException { /* ... */ }

@Transactional(noRollbackFor =
DuplicatePaymentException.class)
public void register(Payment p) { /* ... */ }
```

Rollback-only — метка «транзакцию можно только откатить». Её ставит внутренний `@Transactional`-метод (с `REQUIRED`), если из него вылетело исключение, — даже если внешний метод это исключение поймал:

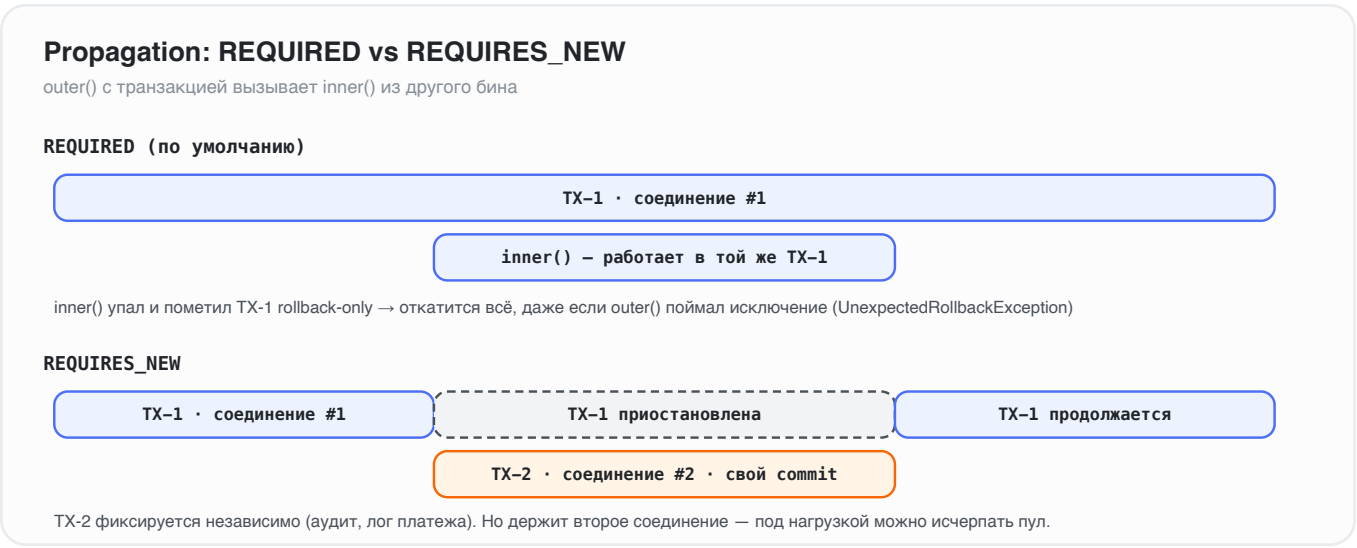
```
@Transactional
public void outer() {
    try {
        auditService.inner();      // @Transactional
(REQUIRED), бросил RuntimeException
    } catch (RuntimeException e) {
        // проглотили
    }
```

```
}   // на commit → UnexpectedRollbackException:
транзакция уже помечена rollback-only
```

Пометить вручную: `TransactionAspectSupport.currentTransactionStatus().setRollbackOnly()`.

Propagation — распространение транзакции

Propagation — что делать, если метод с `@Transactional` вызывается, когда транзакция уже идёт.



Тип	Есть транзакция	Нет транзакции
`REQUIRED` (по умолчанию)	присоединиться	создать
`REQUIRES_NEW`	приостановить, создать новую	создать
`NESTED`	savepoint внутри текущей	создать
`SUPPORTS`	присоединиться	работать без транзакции

Тип	Есть транзакция	Нет транзакции
`NOT_SUPPORTED`	приостановить, работать без	работать без
`MANDATORY`	присоединиться	исключение
`NEVER`	исключение	работать без

```
@Service
class AuditService {
    @Transactional(propagation =
Propagation.REQUIRES_NEW)
    public void logAttempt(UUID paymentId, String
result) {
        // запишется, даже если основной платёж
откатится
    }
}
```

- `REQUIRES_NEW` берёт **второе соединение** из пула. Если все потоки держат по одному соединению и ждут второе — пул исчерпан, приложение встаёт.
- `NESTED` работает через JDBC savepoint (`DataSourceTransactionManager`); с JPA/Hibernate поддержка ограничена.
- `MANDATORY` удобен для методов, которые нельзя вызывать вне транзакции.

Isolation — уровни изоляции

Уровень изоляции — какие изменения параллельных транзакций видны текущей. Чем выше уровень, тем меньше

аномалий, но больше блокировок и ретраев.

Уровень	Dirty read	Non-repeatable read	Phantom read
`READ_UNCOMMITTED`	возможен	возможен	возможен
`READ_COMMITTED`	—	возможен	возможен
`REPEATABLE_READ`	—	—	возможен по стандарту SQL
`SERIALIZABLE`	—	—	—

Особенности PostgreSQL:

- по умолчанию `READ COMMITTED` (`Isolation.DEFAULT` = уровень БД);
- `READ UNCOMMITTED` работает как `READ COMMITTED` — грязного чтения нет;
- `REPEATABLE READ` — снимок данных, фантомов тоже нет; при конкурентном изменении той же строки — ошибка сериализации, нужен ретрай;
- `SERIALIZABLE` — SSI, тоже с ретраями.

```
@Transactional(isolation = Isolation.REPEATABLE_READ)
public Report buildMonthlyReport(YearMonth month) { /*
согласованный снимок */ }
```

Lost update — две транзакции читают баланс 100, обе списывают 30, обе пишут 70 вместо 40. Защита:

```
// 1. атомарный UPDATE — проверка и списание одним запросом
@Modifying
@Query("update Account a set a.balance = a.balance - :amount " +
       "where a.id = :id and a.balance >= :amount")
int withdraw(@Param("id") UUID id, @Param("amount") BigDecimal amount);   // 0 → денег не хватило

// 2. пессимистичная блокировка — SELECT ... FOR UPDATE
// 3. оптимистичная — @Version (см. «Spring Data JPA»)
```

readOnly и timeout

`readOnly = true` — подсказка, а не гарантия:

- Hibernate не делает dirty checking и flush — меньше памяти и CPU на чтении;
- драйвер получает `Connection.setReadOnly(true)` — PostgreSQL отклонит запись в такой транзакции;
- по этому флагу можно направлять запросы на реплику (`AbstractRoutingDataSource`).

Блокировок и изоляции `readOnly` **не даёт**.

```
@Transactional(readOnly = true)
public Page<PaymentView> history(UUID accountId, Pageable page) { /* ... */ }

@Transactional(timeout = 5)            // секунды; дольше → откат
public void settle(UUID batchId) { /* ... */ }
```

Программные транзакции — TransactionTemplate

`TransactionTemplate` — транзакция вокруг куска кода, а не метода. Полезно, когда нужны мелкие границы (каждый элемент цикла в своей транзакции) или нужно обойти self-invocation.

```
@Service
class PayoutService {
    private final TransactionTemplate tx;
    private final PayoutRepository payouts;

    PayoutService(PlatformTransactionManager tm,
PayoutRepository payouts) {
        this.tx = new TransactionTemplate(tm);
        this.payouts = payouts;
    }

    void processBatch(List<UUID> ids) {
        for (UUID id : ids) {
            tx.executeWithoutResult(status -> {
// своя транзакция на каждую выплату

payouts.findById(id).ifPresent(Payout::execute);
            });
        }
    }
}
```

Транзакции и внешний мир — Outbox

Транзакция БД **не распространяется** на HTTP-вызовы, Kafka и почту. Правила:

- не делать сетевых вызовов **внутри** транзакции: соединение с БД висит всё время ожидания, а откатить внешний вызов нельзя;
- записать в БД и отправить в Kafka **атомарно** нельзя — между коммитом и отправкой приложение может упасть.

Transactional Outbox — событие пишется в таблицу `outbox` **в той же транзакции**, что и бизнес-данные. Отдельный процесс (поллер или CDC, например Debezium) читает `outbox` и публикует в Kafka. Консьюмеры идемпотентны: дубль события не создаст второй платёж.

```
@Transactional
public void pay(PaymentCommand cmd) {
    Payment p = payments.save(Payment.from(cmd));
    outbox.save(OutboxEvent.of("payment.completed",
p.getId(), p));
}   // платёж и событие — одна транзакция БД: либо оба,
либо ничего
```

Идемпотентность API — повтор того же запроса не меняет результат. Для платежей клиент шлёт заголовок `Idempotency-Key`, сервер хранит ключ с уникальным индексом и на повтор возвращает сохранённый ответ.

Spring Data JPA

Spring Data JPA — слой доступа к данным поверх JPA/Hibernate: репозитории вместо ручного кода.

Репозитории Spring Data

Spring Data JPA — слой над JPA/Hibernate: ты описываешь интерфейс репозитория, а реализацию (`SimpleJpaRepository`) Spring создаёт сам.

Иерархия: `Repository` → `CrudRepository` / `ListCrudRepository` → `PagingAndSortingRepository` → **`JpaRepository`** (+ `flush` , batch-удаление, `getReferenceById`).

```
@Entity
@Table(name = "accounts")
public class Account {
    @Id
    private UUID id;
    private String ownerEmail;
    private String currency;
    private BigDecimal balance;
    @Version
    private Long version;                    //
оптимистичная блокировка

    protected Account() {}                   // JPA
требует конструктор без аргументов
    // методы withdraw(), deposit() …
}

public interface AccountRepository extends
JpaRepository<Account, UUID> {
    Optional<Account> findByOwnerEmail(String email);
```

```
    List<Account>
findByCurrencyAndBalanceGreaterThan(String currency,
BigDecimal min);
    boolean existsByOwnerEmail(String email);
}
```

- Методы `SimpleJpaRepository` уже транзакционные: класс помечен `@Transactional(readOnly = true)`, изменяющие методы — `@Transactional`.
- Внутри транзакции сервиса управляемую сущность сохранять не нужно — dirty checking сам сделает `UPDATE` при коммите.

Запросы: derived, @Query, проекции, пагинация

Derived query — запрос из имени метода: `findBy…And…OrderBy…`. Удобно для простых условий, для сложных — `@Query`.

```
public interface PaymentRepository extends
JpaRepository<Payment, UUID> {

    // JPQL — по сущностям, а не таблицам
    @Query("select p from Payment p where p.account.id
= :accountId and p.status = :status")
    List<Payment>
findByAccountAndStatus(@Param("accountId") UUID
accountId,

@Param("status") PaymentStatus status);

    // native SQL — когда нужны возможности PostgreSQL
```

```
    @Query(value = "select * from payments where
created_at > now() - interval '1 day'",
           nativeQuery = true)
    List<Payment> findLastDay();

    // изменяющий запрос
    @Modifying(clearAutomatically = true)        //
очистить контекст: в нём устаревшие сущности
    @Query("update Payment p set p.status = 'EXPIRED'
where p.status = 'PENDING' and p.createdAt < :before")
    int expireOlderThan(@Param("before") Instant
before);

    // пагинация
    Page<Payment> findByAccountId(UUID accountId,
Pageable pageable);
}
```

Проекция — выбрать не всю сущность, а нужные поля. Быстрее и без ленивых связей:

```
public record PaymentView(UUID id, BigDecimal amount,
PaymentStatus status) {}

@Query("select new com.bank.payments.PaymentView(p.id,
p.amount, p.status) " +
       "from Payment p where p.account.id =
:accountId")
List<PaymentView> findViews(@Param("accountId") UUID
accountId);
```

`Page` делает дополнительный `count`-запрос; если общее число не нужно — `Slice`.

Проблема N+1

N+1 — один запрос за списком сущностей плюс по одному запросу на каждую, чтобы загрузить ленивую связь.

```
List<Account> accounts = accountRepository.findAll();
// 1 запрос
for (Account a : accounts) {
    a.getPayments().size();
// + N запросов — по одному на счёт
}
```

Решение	Как
`JOIN FETCH`	`select a from Account a join fetch a.paymen`
`@EntityGraph`	`@EntityGraph(attributePaths = "payments")` н репозитория
Batch fetching	`spring.jpa.properties.hibernate.default_bat` — связи грузятся пачками через `IN (…)`
DTO-проекция	загрузить сразу нужные поля без сущностей

```
@EntityGraph(attributePaths = "payments")
List<Account> findByCurrency(String currency);
```

⚠ JOIN FETCH коллекции + пагинация

Hibernate не может применить `LIMIT` к строкам join-а и пагинирует **в памяти** — загрузит всё и предупредит в логе. Для страниц: сначала id страницы, потом fetch по этим id, или batch fetching.

Как найти N+1: `logging.level.org.hibernate.SQL=debug` и смотреть число запросов, в тестах — считать запросы (Hibernate statistics).

Ленивая загрузка и open-in-view

Lazy loading — связь грузится при первом обращении, а не вместе с сущностью. Для `@OneToMany` / `@ManyToMany` по умолчанию `LAZY`, для `@ManyToOne` / `@OneToOne` — `EAGER` (лучше явно ставить `LAZY`).

`LazyInitializationException` — обращение к ленивой связи, когда сессия Hibernate уже закрыта (вне транзакции).

Open Session in View (`spring.jpa.open-in-view`, по умолчанию **true**, Boot пишет предупреждение) держит сессию открытой до конца HTTP-запроса — ленивые связи «работают» и в контроллере. Минусы: скрытые запросы при сериализации JSON, соединение с БД занято дольше.

```
spring:
  jpa:
    open-in-view: false     # данные загружаем явно в
сервисе: fetch join, EntityGraph, DTO
```

save(): persist или merge

`SimpleJpaRepository.save()` решает по `isNew()`:

- новая сущность → `persist` (только `INSERT`);
- иначе → `merge` (сначала `SELECT`, потом `UPDATE` или `INSERT`).

Сущность новая, если `@Id` равен `null` — или `@Version` равен `null`, если поле версии есть.

> ⚠ **UUID, который генерируешь сам**
>
> Если `id = UUID.randomUUID()` задан до `save()` и нет `@Version`, Spring считает сущность **не новой** → лишний `SELECT` перед каждой вставкой. Решения: `@Version Long version` (обёртка — `null` у новой) или реализовать `Persistable<UUID>` с собственным `isNew()`.

Блокировки: optimistic и pessimistic

Оптимистичная блокировка — конфликт проверяется при записи по полю `@Version`: `UPDATE … WHERE id = ? AND version = ?`. Если строку уже изменили — `ObjectOptimisticLockingFailureException`, операцию повторяют.

Пессимистичная блокировка — строка блокируется при чтении (`SELECT … FOR UPDATE`), другие транзакции ждут.

```
public interface AccountRepository extends
JpaRepository<Account, UUID> {

    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("select a from Account a where a.id = :id")
    Optional<Account> findByIdForUpdate(@Param("id")
UUID id);   // SELECT … FOR UPDATE
}
```

	Optimistic `@Version`	**Pessimistic `FOR UPDATE`**
Конфликты	редкие	частые (горячие счета)
Цена	ретрай при конфликте	ожидание блокировки, риск дедлока
Пример	профиль пользователя	списание с одного счёта параллельными платежами

> **Против дедлоков**
>
> При переводе между двумя счетами блокируй их **в одном порядке** — например, по возрастанию id. Иначе два встречных перевода заблокируют друг друга.

Аудит сущностей

Аудит — автоматическое заполнение служебных полей: когда и кем создано или изменено.

```
@Configuration
@EnableJpaAuditing
class JpaConfig {}

@Entity
@EntityListeners(AuditingEntityListener.class)
public class Payment {
    @CreatedDate      private Instant createdAt;
    @LastModifiedDate private Instant updatedAt;
    @CreatedBy        private String createdBy;    //
```

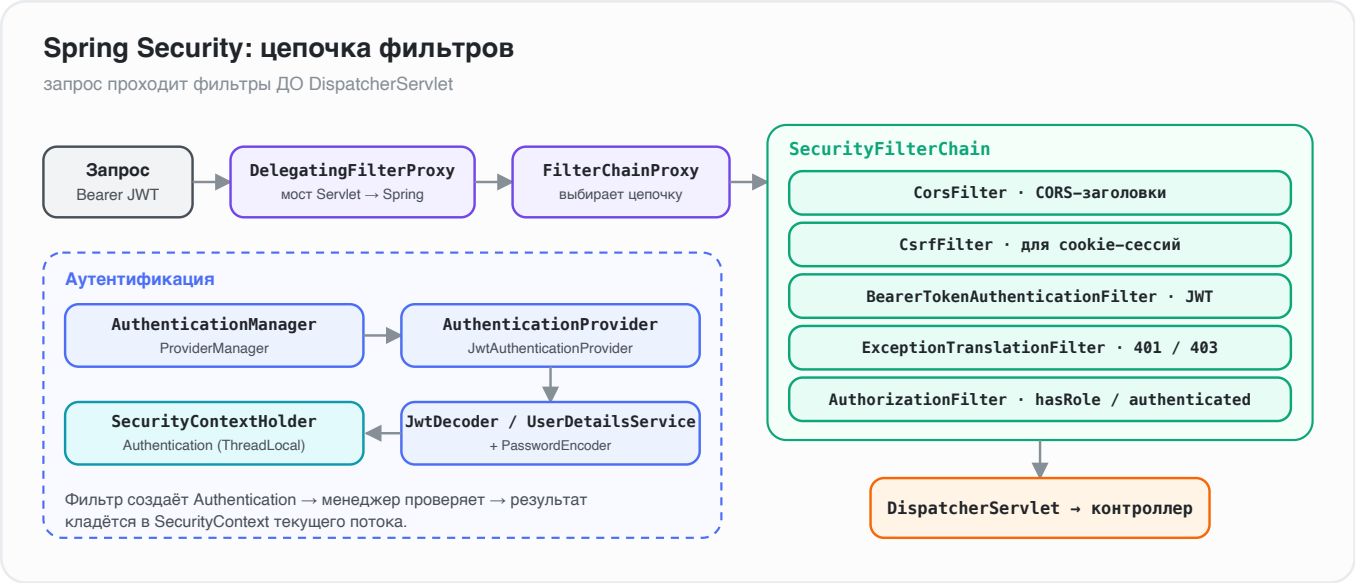
```
нужен бин AuditorAware<String>
}
```

Spring Security

Spring Security — аутентификация и авторизация через цепочку фильтров.

Архитектура: цепочка фильтров

Spring Security — фреймворк аутентификации и авторизации. В веб-приложении он работает как **цепочка Servlet-фильтров**, через которую запрос проходит **до** `DispatcherServlet`.



Компонент	Роль
`DelegatingFilterProxy`	обычный Servlet-фильтр, передаёт управление в Spring
`FilterChainProxy`	выбирает подходящую `SecurityFilterChain` по URL

Компонент	Роль
`SecurityFilterChain`	упорядоченный набор фильтров безопасности; цепочек может быть несколько (`/api/**` , `/actuator/**`)
`AuthenticationManager` (`ProviderManager`)	проверяет учётные данные через список `AuthenticationProvider`
`SecurityContextHolder`	хранит `Authentication` текущего запроса — по умолчанию в `ThreadLocal`

Настройка SecurityFilterChain

`SecurityFilterChain` настраивается бином через лямбда-DSL. `WebSecurityConfigurerAdapter` удалён ещё в Security 6, а в Security 7 остался только лямбда-стиль.

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
class SecurityConfig {

    @Bean
    SecurityFilterChain api(HttpSecurity http) throws
Exception {
        http
            .csrf(csrf -> csrf.disable())
// stateless API с Bearer-токеном
            .cors(Customizer.withDefaults())
            .sessionManagement(s ->
s.sessionCreationPolicy(SessionCreationPolicy.STATELESS
))
```

```
            .authorizeHttpRequests(auth -> auth

.requestMatchers("/actuator/health/**").permitAll()
                .requestMatchers(HttpMethod.POST,
"/api/payments/**").hasAuthority("SCOPE_payments:write"
)

.requestMatchers("/api/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated())
            .oauth2ResourceServer(oauth ->
oauth.jwt(Customizer.withDefaults()));
        return http.build();
    }
}
```

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri:
https://auth.example.com/realms/bank   # ключи для
проверки подписи — отсюда
```

Правила `authorizeHttpRequests` проверяются **сверху вниз** — первое подходящее срабатывает. Поэтому `anyRequest()` всегда последний.

Аутентификация и авторизация

Аутентификация (AuthN) — кто ты: проверка логина и пароля, токена, сертификата.

Авторизация (AuthZ) — что тебе можно: проверка прав на действие или ресурс.

`Authentication` содержит:

- `principal` — пользователь;
- `credentials` — пароль или токен, после проверки обычно стираются;
- `authorities` — права (`GrantedAuthority`).

Понятие	Как устроено
Authority	строка-право: `payments:write` , `SCOPE_read`
Role	authority с префиксом `ROLE_` : `hasRole("ADMIN")` проверяет `ROLE_ADMIN`
Scope из JWT	по умолчанию становится authority с префиксом `SCOPE_`

```
@GetMapping("/me")
UserInfo me(@AuthenticationPrincipal Jwt jwt) {
    return new UserInfo(jwt.getSubject(),
jwt.getClaimAsString("email"));
}
```

⚠ SecurityContext живёт в ThreadLocal

В `@Async` -методе или в своём `ExecutorService` контекст пустой. Нужна обёртка `DelegatingSecurityContextExecutor` или явная передача данных пользователя.

Method security

Method security — проверка прав на уровне методов сервисов, а не URL. Включается `@EnableMethodSecurity`, работает через прокси — с той же ловушкой self-invocation.

```
@Service
class AccountService {

    @PreAuthorize("hasRole('ADMIN')")
    public void freeze(UUID accountId) { /* ... */ }

    @PreAuthorize("@accountAccess.isOwner(#accountId,
authentication)")
    public Balance balance(UUID accountId) { /* ... */
}
}

@Component("accountAccess")
class AccountAccess {
    boolean isOwner(UUID accountId, Authentication
auth) { /* счёт принадлежит auth.getName()? */ return
true; }
}
```

⚠ Проверка владельца — обязательна

`GET /accounts/{id}` с чужим id — классическая уязвимость IDOR. Мало проверить, что пользователь аутентифицирован: нужно проверить, что **этот объект — его**.

JWT

JWT (JSON Web Token) — токен из трёх частей `header.payload.signature` в Base64URL. Он **подписан, но не зашифрован**: payload может прочитать любой.

Что проверяет resource server:

- подпись — по публичному ключу из JWKS по `issuer-uri`;
- `exp` — не истёк ли токен;
- `iss` — кто выпустил;
- при необходимости `aud` — для кого токен.

Практика	Почему
Короткий access token (5–15 мин) + refresh token	JWT нельзя отозвать до истечения срока
Никаких секретов в payload	его может прочитать кто угодно
Асимметричная подпись (RS256/ES256)	сервисы проверяют токен, но не могут его выпустить
Выпуск токенов — отдельный сервер авторизации	Keycloak, Spring Authorization Server

Хранение паролей

`PasswordEncoder` — однонаправленное медленное хеширование с солью. Пароли никогда не хранят открытым текстом и не хешируют быстрыми MD5/SHA-256.

```
@Bean
PasswordEncoder passwordEncoder() {
    return
PasswordEncoderFactories.createDelegatingPasswordEncode
```

```
r();   // bcrypt по умолчанию
}
```

Хеш хранится с префиксом алгоритма: `{bcrypt}$2a$10$…` . Это позволяет позже перейти на Argon2, не ломая старые пароли.

CSRF и CORS

CSRF — атака, при которой чужой сайт заставляет браузер отправить запрос с **cookie** пользователя. Защита нужна, если аутентификация идёт через cookie/сессию. Для stateless API с токеном в заголовке `Authorization` браузер токен сам не подставит — CSRF можно отключить.

CORS — политика **браузера**: разрешает ли сайт с другого домена читать ответы твоего API. Это **не** аутентификация и не защита от запросов вне браузера.

```
@Bean
CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration c = new CorsConfiguration();

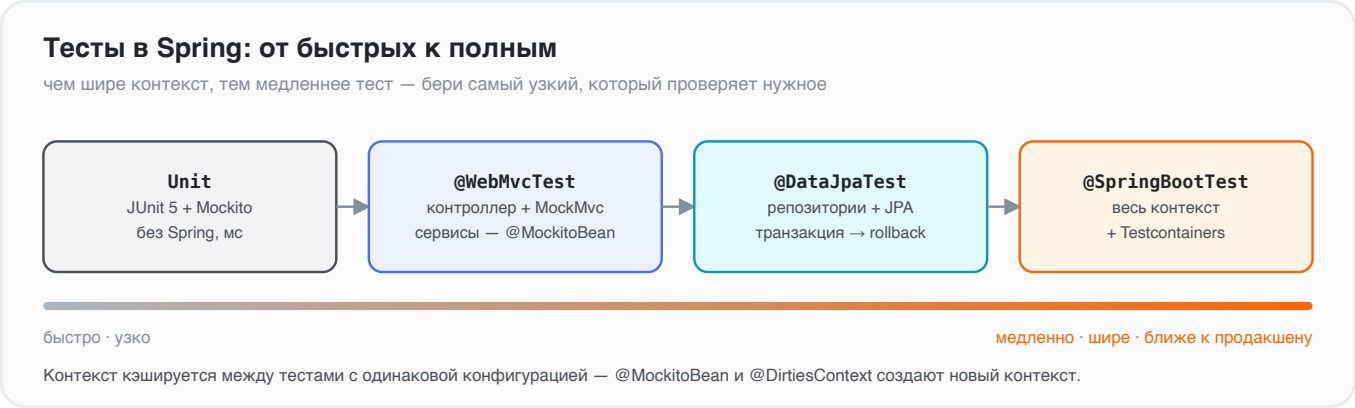
c.setAllowedOrigins(List.of("https://app.example.com"))
;
    c.setAllowedMethods(List.of("GET", "POST"));
    c.setAllowedHeaders(List.of("Authorization",
"Content-Type", "Idempotency-Key"));
    UrlBasedCorsConfigurationSource source = new
UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/api/**", c);
    return source;
}
```

Spring Testing

Тестирование в Spring — от быстрых unit-тестов до полного контекста с настоящей БД.

Виды тестов в Spring

Тестовый срез (test slice) — аннотация, которая поднимает не весь контекст, а только нужный слой: веб, JPA, JSON. Такой тест быстрее и точнее показывает, что сломалось.



Тест	Что поднимается	Для чего
Unit (JUnit 5 + Mockito)	ничего из Spring	бизнес-логика сервисов
`@WebMvcTest`	контроллеры, фильтры, `MockMvc`, Jackson, валидация	маппинг, статусы, JSON, ошибки
`@DataJpaTest`	JPA, репозитории, `DataSource`	запросы, маппинг сущностей
`@SpringBootTest`	весь контекст	сквозной сценарий, интеграция компонентов

Unit-тесты: JUnit 5 + Mockito

Unit-тест проверяет один класс в изоляции: зависимости заменяются моками, Spring не запускается — тесты идут миллисекунды.

```
@ExtendWith(MockitoExtension.class)
class TransferServiceTest {

    @Mock AccountRepository accounts;
    @InjectMocks TransferService service;

    static final UUID FROM = UUID.randomUUID();
    static final UUID TO = UUID.randomUUID();

    @Test
    void rejectsTransferWhenNotEnoughMoney() {

when(accounts.findByIdForUpdate(FROM)).thenReturn(Optio
nal.of(new Account(FROM, new BigDecimal("10.00"))));

when(accounts.findByIdForUpdate(TO)).thenReturn(Optiona
l.of(new Account(TO, BigDecimal.ZERO)));

        assertThatThrownBy(() -> service.transfer(FROM,
TO, new BigDecimal("50.00")))

.isInstanceOf(InsufficientFundsException.class);
    }
}
```

Constructor injection делает unit-тесты простыми: сервис можно собрать через `new TransferService(mock)` без Spring.

@WebMvcTest — тест контроллера

`@WebMvcTest` поднимает только веб-слой. Сервисы подменяются `@MockitoBean`, запросы отправляются через `MockMvc` без реального HTTP.

```
@WebMvcTest(PaymentController.class)
class PaymentControllerTest {

    @Autowired MockMvc mvc;
    @MockitoBean PaymentService service;
// вместо @MockBean (устарел в Boot 3.4)

    @Test
    @WithMockUser
    void rejectsNegativeAmount() throws Exception {
        mvc.perform(post("/api/payments")
                .with(csrf())
                .header("Idempotency-Key", "k-1")

.contentType(MediaType.APPLICATION_JSON)
                .content("""

{"fromAccount":"%s","toAccount":"%s","amount":-5,"curre
ncy":"EUR"}
                    """.formatted(UUID.randomUUID(),
UUID.randomUUID())))
           .andExpect(status().isBadRequest());

        verifyNoInteractions(service);
// до сервиса запрос не дошёл
    }
}
```

- Если в проекте есть Spring Security, `@WebMvcTest` применяет её: нужен `@WithMockUser` или `.with(jwt())`; при включённом CSRF — `.with(csrf())`.
- В Spring 6.2 появился `MockMvcTester` — тот же `MockMvc`, но с проверками в стиле AssertJ.

@DataJpaTest и Testcontainers

`@DataJpaTest` поднимает только JPA и репозитории. Каждый тест выполняется в транзакции и **откатывается** в конце.
Testcontainers запускает настоящий PostgreSQL в Docker. Это точнее встроенной H2: те же типы, индексы и SQL-диалект, что в продакшене.
`@ServiceConnection` (Boot 3.1+) сам передаёт адрес контейнера в настройки `DataSource`.

```
@DataJpaTest
@AutoConfigureTestDatabase(replace =
AutoConfigureTestDatabase.Replace.NONE)   // не
подменять на H2
@Testcontainers
class AccountRepositoryTest {

    @Container
    @ServiceConnection
    static PostgreSQLContainer<?> postgres = new
PostgreSQLContainer<>("postgres:17");

    @Autowired AccountRepository accounts;

    @Test
    void findsAccountByOwnerEmail() {
```

```
        accounts.save(new Account(UUID.randomUUID(),
"ann@bank.com", "EUR", BigDecimal.TEN));


assertThat(accounts.findByOwnerEmail("ann@bank.com")).i
sPresent();
    }
}
```

Имя пакета `PostgreSQLContainer` зависит от версии Testcontainers: в 2.x модули и пакеты переименованы.

@SpringBootTest — интеграционный тест

`@SpringBootTest` поднимает весь контекст приложения — как в продакшене, только с тестовыми настройками.

```
@SpringBootTest
@AutoConfigureMockMvc
@Import(TestcontainersConfig.class)
class PaymentFlowIT {

    @Autowired MockMvc mvc;

    @Test
    @WithMockUser(authorities = "SCOPE_payments:write")
    void createsPayment() throws Exception {
        mvc.perform(post("/api/payments") /* ... */)
           .andExpect(status().isCreated());
    }
}

@TestConfiguration(proxyBeanMethods = false)
```

```
class TestcontainersConfig {
    @Bean
    @ServiceConnection
    PostgreSQLContainer<?> postgres() { return new
PostgreSQLContainer<>("postgres:17"); }
}
```

- **Кэш контекста:** тесты с одинаковой конфигурацией переиспользуют один контекст. Разные наборы `@MockitoBean`, профилей и `@DirtiesContext` создают новый контекст — тесты замедляются.
- Общая `@TestConfiguration` с контейнерами — один PostgreSQL на все интеграционные тесты.

Ловушки транзакций в тестах

`@Transactional` на тесте (и `@DataJpaTest` по умолчанию) откатывает всё в конце. Удобно, но скрывает проблемы:

Что скрывается	Почему
`LazyInitializationException`	в тесте сессия открыта весь метод, в продакшене — нет
Нарушение ограничений БД	`INSERT` не выполнился: не было `flush` — вызывай `flush()`

Что скрывается	Почему
`@TransactionalEventListener(AFTER_COMMIT)`	коммита нет — слушатель не вызовется
`REQUIRES_NEW`	эта транзакция **реально коммитится** и не откатывается вместе с тестом

Для сценариев «как в продакшене» — тест без `@Transactional` и очистка данных между тестами.

Вопросы к собеседованию

Spring Core — вопросы

Что такое IoC и DI?

IoC — принцип: объекты не создают зависимости сами, этим управляет контейнер. DI — способ реализации IoC: контейнер передаёт зависимости через конструктор, сеттер или поле.

Какой способ DI лучше и почему?

Через конструктор: поля `final`, объект нельзя создать в неполном состоянии, легко тестировать без Spring, циклы видны сразу.

Чем ApplicationContext отличается от BeanFactory?

`ApplicationContext` расширяет `BeanFactory`: создаёт singleton-бины сразу при старте, сам регистрирует `BeanPostProcessor`-ы, умеет события, i18n, интеграцию с AOP. `BeanFactory` — минимальный контейнер с ленивым созданием.

Какие бывают scope бинов?

`singleton` (по умолчанию), `prototype`, а в веб-приложениях — `request`, `session`, `application`, `websocket`.

Потокобезопасен ли singleton-бин?

Нет. Один объект на все потоки — код должен быть stateless или использовать потокобезопасные структуры. Подробнее: «Потокобезопасность singleton-бинов».

Как внедрить prototype в singleton, чтобы каждый раз был новый объект?

`ObjectProvider<T>`, `@Lookup` или scoped proxy. Подробнее: «Prototype внутри singleton и scoped proxy».

Что такое BeanPostProcessor и BeanFactoryPostProcessor?

BFPP меняет описания бинов до их создания, BPP обрабатывает готовые объекты до и после инициализации и может подменить их прокси.

В каком порядке вызываются init-методы?

`@PostConstruct` → `afterPropertiesSet()` → `initMethod`. Для уничтожения: `@PreDestroy` → `destroy()` → `destroyMethod`. У prototype-бинов destroy-колбэки Spring не вызывает.

JDK-прокси или CGLIB — в чём разница?

JDK — через интерфейсы, CGLIB — через наследование класса. Spring Boot по умолчанию использует CGLIB. CGLIB не перехватывает `final` и `private` методы.

Почему аннотация не срабатывает при вызове метода из того же класса?

Self-invocation: `this.method()` вызывает настоящий объект, а не прокси. Решение — вынести метод в другой бин.

Чем `@Configuration` отличается от `@Component` с `@Bean`-методами?

`@Configuration` проксируется, и вызов `@Bean`-метода возвращает singleton из контекста. В `@Component` это обычный вызов — получится новый объект.

Что будет при циклической зависимости?

С Spring Boot 2.6 приложение не стартует. Лечится перепроектированием, временно — `@Lazy`.

`@Primary` или `@Qualifier`?

`@Primary` — бин по умолчанию среди нескольких кандидатов. `@Qualifier` — точный выбор в месте внедрения, он сильнее `@Primary`.

Spring Boot — вопросы

Чем Spring Boot отличается от Spring?

Spring — фреймворк, Boot — способ быстро собрать на нём приложение: автоконфигурация, стартеры, встроенный сервер, Actuator, внешняя конфигурация.

Из чего состоит `@SpringBootApplication`?

`@SpringBootConfiguration` + `@EnableAutoConfiguration` + `@ComponentScan` (пакет главного класса и вложенные).

Как работает автоконфигурация?

Boot читает список классов из `AutoConfiguration.imports` во всех jar-ах и применяет те, чьи `@Conditional…`-условия выполнены. Подробнее: «Автоконфигурация».

Как переопределить бин из автоконфигурации?

Объявить свой бин того же типа: автоконфиг помечен `@ConditionalOnMissingBean` и отступит.

Как понять, почему бин не создался?

Запустить с `--debug` и посмотреть Condition Evaluation Report или открыть `/actuator/conditions`.

Что такое стартер?

Зависимость, которая подтягивает набор библиотек и их автоконфигурацию с согласованными версиями.

Какой приоритет у источников настроек?

Командная строка → системные свойства → переменные окружения → профильный `application-{profile}.yml` → `application.yml`.

`@Value` или `@ConfigurationProperties`?

Для группы настроек — `@ConfigurationProperties`: типизация, валидация на старте, гибкое связывание. `@Value` — для одиночного значения или SpEL.

Что такое liveness и readiness?

Liveness — живо ли приложение (если нет — перезапуск пода). Readiness — готово ли принимать трафик (если нет — под временно убирают из балансировки).

Что такое graceful shutdown?

Остановка с дожиданием текущих запросов. В Boot 3.4+ включён по умолчанию, время ожидания — `spring.lifecycle.timeout-per-shutdown-phase`.

Как включить виртуальные потоки и что может пойти не так?

`spring.threads.virtual.enabled=true`. Узкое место — пул соединений к БД; до Java 24 — pinning на `synchronized`.

Что нового в Spring Boot 4?

Java 17+/25, Jakarta EE 11, модульность, JSpecify, версионирование API, HTTP service clients, Jackson 3, Spring

Security 7.

Spring MVC и REST — вопросы

Как запрос доходит до метода контроллера?

Фильтры → `DispatcherServlet` → `HandlerMapping` находит метод → `HandlerAdapter` готовит аргументы → интерсепторы → метод → `HttpMessageConverter` пишет ответ. Подробнее: «DispatcherServlet и путь запроса».

`@Controller` vs `@RestController`?

`@RestController` добавляет `@ResponseBody` — ответ сериализуется в тело, без представления (шаблона).

Как обработать исключения централизованно?

`@RestControllerAdvice` + `@ExceptionHandler`, ответ — `ProblemDetail`.

Как валидировать входные данные?

Ограничения Bean Validation на DTO + `@Valid` на `@RequestBody`; ошибка → 400.

Чем Filter отличается от Interceptor?

Filter — уровень Servlet-контейнера, до Spring MVC. Interceptor — внутри `DispatcherServlet` и знает, какой метод контроллера будет вызван.

Какие HTTP-методы идемпотентны?

`GET`, `PUT`, `DELETE` (и `HEAD`, `OPTIONS`). `POST` — нет, `PATCH` — не обязательно.

Как сделать POST платежа идемпотентным?

Ключ `Idempotency-Key` от клиента, уникальный индекс в БД, на повтор — сохранённый ответ.

401 vs 403?

401 — не аутентифицирован (нет или неверный токен). 403 — аутентифицирован, но нет прав.

`PUT` vs `PATCH`?

`PUT` заменяет ресурс целиком, `PATCH` меняет часть полей.

`RestTemplate`, `RestClient` или `WebClient`?

В новом синхронном коде — `RestClient` или HTTP interface, в реактивном — `WebClient`. `RestTemplate` — для старого кода.

Когда нужен WebFlux?

Очень много одновременных соединений или стриминг. В остальных случаях — MVC, особенно с виртуальными потоками.

Spring Transactions — вопросы

Как работает `@Transactional` под капотом?

Прокси вызывает `TransactionInterceptor`, тот через `PlatformTransactionManager` открывает транзакцию, привязывает соединение к потоку, вызывает метод, затем commit или rollback.

Почему `@Transactional` может не сработать?

Self-invocation, `private` / `final`-метод, объект не бин, исключение поймано внутри, checked-исключение, работа в другом потоке. Подробнее: «Почему @Transactional не работает».

На каких исключениях откатывается транзакция?

На `RuntimeException` и `Error`. На checked — commit, если не указать `rollbackFor`.

Что такое `UnexpectedRollbackException`?

Внутренний метод с `REQUIRED` пометил общую транзакцию rollback-only, а внешний поймал исключение и попытался закоммитить.

`REQUIRED` vs `REQUIRES_NEW`?

`REQUIRED` присоединяется к текущей транзакции, `REQUIRES_NEW` приостанавливает её и открывает новую на другом соединении — коммитится независимо, но может исчерпать пул.

Что такое `NESTED`?

Вложенная транзакция через savepoint: откат вложенной части не откатывает внешнюю. Работает с JDBC, с JPA ограниченно.

Какие уровни изоляции и какие аномалии они предотвращают?

Read uncommitted → read committed (нет dirty read) → repeatable read (нет non-repeatable read) → serializable (нет phantom). В PostgreSQL по умолчанию read committed.

Что такое lost update и как защититься?

Две транзакции перезаписывают изменения друг друга. Атомарный `UPDATE` с условием, `SELECT … FOR UPDATE` или `@Version`.

Что даёт `readOnly = true`?

Оптимизацию Hibernate (без dirty checking и flush) и read-only соединение. Блокировок и изоляции не даёт.

Можно ли в одной транзакции записать в БД и отправить сообщение в Kafka?

Атомарно — нет. Используют Transactional Outbox и идемпотентных консьюмеров.

Передаётся ли транзакция в `@Async`-метод?

Нет: транзакция привязана к потоку, у `@Async` поток другой.

Почему нельзя делать HTTP-вызов внутри транзакции?

Соединение с БД занято всё время ожидания ответа, а откатить внешний вызов при rollback невозможно.

Spring Data JPA — вопросы

Как Spring создаёт реализацию репозитория?

При старте генерирует прокси на базе `SimpleJpaRepository`, а derived-методы разбирает по имени и строит JPQL.

`CrudRepository` vs `JpaRepository`?

`JpaRepository` расширяет CRUD + пагинацию и добавляет JPA-специфику: `flush`, batch-удаление, `getReferenceById`.

Что такое N+1 и как его решить?

Лишние запросы на ленивые связи в цикле. `JOIN FETCH`, `@EntityGraph`, batch fetching, DTO-проекции. Подробнее: «Проблема N+1».

Откуда `LazyInitializationException`?

Ленивая связь читается вне открытой сессии Hibernate. Загружать нужное явно внутри транзакции.

Что такое open-in-view и почему его выключают?

Сессия открыта на весь HTTP-запрос: скрытые запросы в контроллере и при сериализации, соединение занято дольше.

Как `save()` отличает новую сущность?

По `null` в `@Id` или `@Version`. Иначе вызывается `merge` с лишним `SELECT`.

Нужно ли вызывать `save()` для изменённой сущности в транзакции?

Нет: управляемая сущность сохранится при коммите через dirty checking.

Optimistic vs pessimistic locking?

Optimistic — проверка `@Version` при записи и ретрай, когда конфликты редки. Pessimistic — `SELECT … FOR UPDATE`, когда конфликты частые.

Зачем `@Modifying` и `clearAutomatically`?

`@Modifying` нужен для `UPDATE`/`DELETE` в `@Query`. `clearAutomatically` очищает контекст, где после массового обновления остались устаревшие сущности.

`Page` vs `Slice`?

`Page` знает общее число элементов (лишний `count`), `Slice` — только есть ли следующая страница.

Spring Security — вопросы

Как устроен Spring Security в веб-приложении?

Цепочка фильтров: `DelegatingFilterProxy` → `FilterChainProxy` → `SecurityFilterChain`. Фильтры аутентифицируют запрос, кладут `Authentication` в `SecurityContextHolder`, `AuthorizationFilter` проверяет права. Подробнее: «Архитектура: цепочка фильтров».

Аутентификация vs авторизация?

Аутентификация — кто ты, авторизация — что тебе можно.

401 vs 403?

401 — нет или неверная аутентификация, 403 — аутентифицирован, но прав нет. Оба формирует

`ExceptionTranslationFilter`.

Role vs authority?

Role — authority с префиксом `ROLE_`. `hasRole("ADMIN")` = `hasAuthority("ROLE_ADMIN")`.

Как настроить безопасность в Spring Security 6/7?

Бин `SecurityFilterChain` с лямбда-DSL. `WebSecurityConfigurerAdapter` удалён.

Где хранится текущий пользователь?

В `SecurityContextHolder`, по умолчанию в `ThreadLocal` — в других потоках его нет.

Что такое JWT и как его проверяют?

Подписанный токен `header.payload.signature`. Проверяют подпись, срок `exp` и издателя `iss`. Отозвать до истечения нельзя — поэтому короткий срок жизни.

Нужно ли отключать CSRF?

Для stateless API с Bearer-токеном — можно. Для cookie-сессий — нельзя.

Чем CORS отличается от CSRF?

CORS — разрешение браузеру читать ответы с другого домена. CSRF — атака через автоматически отправленные cookie. Это разные механизмы.

Как хранить пароли?

Через `PasswordEncoder` с bcrypt или Argon2 — медленный хеш с солью.

Что такое IDOR и как защититься?

Доступ к чужому объекту подменой id. Проверять владельца объекта: `@PreAuthorize` с проверкой или фильтр в запросе по текущему пользователю.

Как работает `@PreAuthorize`?

Через прокси метод-секьюрити: SpEL-выражение вычисляется до вызова метода. Нужен `@EnableMethodSecurity`.

Spring Testing — вопросы

Какие виды тестов есть в Spring и когда какой брать?

Unit — логика без Spring; `@WebMvcTest` — контроллеры; `@DataJpaTest` — репозитории; `@SpringBootTest` — весь контекст. Брать самый узкий, который проверяет нужное.

`@MockBean` vs `@MockitoBean`?

`@MockitoBean` из Spring Framework 6.2 — замена `@MockBean`, который устарел в Spring Boot 3.4. Оба кладут мок в контекст вместо настоящего бина.

`@Mock` vs `@MockitoBean`?

`@Mock` — чистый Mockito в unit-тесте без Spring. `@MockitoBean` заменяет бин в контексте Spring.

Зачем Testcontainers, если есть H2?

H2 — другая СУБД: другой SQL, типы, поведение блокировок. Testcontainers даёт настоящий PostgreSQL.

Что такое кэш контекста и что его ломает?

Spring переиспользует контекст между тестами с одинаковой конфигурацией. Новый контекст создают разные `@MockitoBean`, профили, свойства и `@DirtiesContext`.

Почему `@Transactional` в тестах может обмануть?

Откат в конце скрывает ошибки flush и коммита, ленивую загрузку и слушателей `AFTER_COMMIT`. Подробнее: «Ловушки транзакций в тестах».

Как протестировать контроллер с Spring Security?

`@WebMvcTest` + `@WithMockUser` или `.with(jwt())`, при CSRF — `.with(csrf())`.

Что такое `@ServiceConnection`?

Аннотация Boot 3.1+: берёт адрес и логин из Testcontainers-контейнера и настраивает подключение без `@DynamicPropertySource`.